

[1]

1

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
“ВЫСШАЯ ШКОЛА ЭКОНОМИКИ”
ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ НАУК**

Вальсамакин Александр Александрович
Удалов Даниил Григорьевич

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

Улучшение генерации музыки с помощью обучения с подкреплением

Enhanced Music Generation via Reinforcement Learning

по направлению подготовки 01.04.02 Прикладная математика и информатика
образовательная программа «Искусственный интеллект»

Студенты

А.А. Вальсамакин

Д.Г. Удалов

Научный руководитель

к.ф.-м.н., доцент

Е.О. Кантонистова

Соруководитель

Департамент больших данных и информационного поиска

П.М. Гринберг

Москва 2026

Содержание

Аннотация

В работе рассматривается задача улучшения генерации MIDI-фрагментов по текстовому описанию. Базовая text-to-MIDI модель способна порождать синтаксически корректные музыкальные последовательности, однако стандартное контролируемое обучение не задаёт напрямую критерии качества готового фрагмента: тональную согласованность, устойчивость ритма, осмысленную плотность нот, соотношение мелодии и аккомпанемента, а также соответствие музыкального результата текстовому запросу.

Для решения этой проблемы исследуется схема последующего дообучения модели с использованием GRPO. Для одного текстового запроса генерируется группа MIDI-кандидатов, после чего они оцениваются не по отдельным токенам, а как завершённые музыкальные фрагменты. В качестве обучающего сигнала используются две группы наград: правила, основанные на музыкально-теоретических и статистических признаках, и оценка внешнего графового критика. Правила учитывают тональность, ритмическую структуру, плотность нот, долю ударных событий и другие свойства MIDI. Графовый критик строится отдельно: музыкальный фрагмент преобразуется в гетерогенный граф, в котором вершины соответствуют нотам, аккордам, тактам и временным позициям, а рёбра отражают временные и структурные связи. Для обучения критика используется корпус HookTheory Dataset, содержащий разметку мелодии, гармонии, тональности и метрической структуры. Рассматривается схема из графового учителя и графового ученика: учитель обучается различать корректные и искажённые музыкальные графы, а ученик приближает его оценку по признакам, извлекаемым из MIDI.

Экспериментальная часть включает несколько режимов дообучения: обучение по правилам, обучение по графовому критику, а также комбинированный сценарий, в котором критик применяется после предварительного rule-based этапа. Качество моделей анализируется по внутренним наградам, A/B-сравнениям с базовой моделью и внешней оценке языковой модели, работающей с ABC-представлением MIDI. Полученные результаты показывают, что последующее дообучение может смещать распределение генераций в сторону более предпочтительных по заданному критерию фрагментов, однако эффект зависит от качества награды, выбора запросов и устойчивости самого критика.

Ключевые слова: text-to-MIDI, символическая музыка, большие языковые модели, обучение с подкреплением, GRPO, графовые нейронные сети.

Введение

Генерация музыки по текстовому описанию находится на пересечении обработки естественного языка, символьной музыкальной генерации и задач управляемой музыкальной генерации. В такой постановке модель должна не просто сгенерировать правдоподобную последовательность нот, но и согласовать музыкальный результат с текстовым запросом: жанром, настроением, фактурой, инструментальным составом и структурой [??], [??], [??]. Именно поэтому задача *text-to-MIDI* существенно сложнее обычной безусловной генерации: здесь нужно одновременно моделировать музыкальную последовательность и межмодальное соответствие текста и музыки.

Актуальность темы определяется двумя обстоятельствами. Во-первых, крупные языковые модели сделали реалистичным перенос методов дообучения по инструкциям и оптимизации по предпочтениям в музыкальную область [??], [??], [??]. Во-вторых, даже сильные генераторы символьной музыки всё ещё слабо гарантируют гармоническую согласованность, развитие материала и устойчивую многодорожечную структуру [??], [??], [??], [??]. Поэтому естественным направлением является дообучение модели по внешним критериям качества, а не только по цели максимизации вероятности обучающих последовательностей.

Цель работы состоит в исследовании возможности улучшить генерацию *MIDI* по текстовому описанию с помощью обучения с подкреплением (*RL*). Для этого необходимо: кратко описать релевантные формы представления музыкальных данных; выделить сильные и слабые стороны современных генеративных архитектур; рассмотреть существующие решения для генерации *MIDI* по тексту; проанализировать метрики; и обосновать собственную схему дообучения, сочетающую *Text2midi*, *Group Relative Policy Optimization (GRPO)* и структурный графовый критик.

За выбор набора обучающих данных и обучение графового критика ответственен А. А. Вальсамакин

За дообучение *text2midi* модели и внедрение критика ответственен Д. Г. Удалов

Обзор литературы

Символьное представление музыки

Для задач композиционного моделирования наиболее естественным остаётся символьное представление, поскольку оно явно кодирует ноты, длительности, метрические позиции, темп, аккорды и структуру партий, то есть именно те сущности, которые важны при анализе гармонии и формы [??], [??], [??]. В отличие от звукового сигнала, символьные данные меньше зависят от тембра и студийной обработки и лучше подходят для композиционного контроля. Среди символьных форматов используются фортепианная матрица, ABC-нотация, *MIDI* и его расширения; для практических генеративных систем именно *MIDI* остаётся наиболее универсальным представлением благодаря стандартизованному событийно-временному формату и поддержке многодорожечности [??], [??], [??].

Для генерации *MIDI* по текстовому описанию это особенно важно. Если ABC-нотация удобна для применения языковых моделей как к обычному тексту, то *MIDI* ближе к прикладной задаче: он допускает несколько инструментов, пригоден для воспроизведения, анализа и последующего редактирования. Поэтому обзор работ на ABC-нотации важен как обзор возможных методов адаптации крупных языковых моделей, но прикладная цель работы остаётся связанной именно с генерацией *MIDI*-последовательностей [??], [??], [??].

Основные архитектуры генерации символьной музыки и их ограничения

Современная музыкальная генерация опирается прежде всего на трансформеры, вариационные автокодировщики, генеративно-состязательные сети, диффузионные модели и крупные предварительно обученные языковые модели [??], [??], [??]. Трансформеры сегодня являются наиболее универсальным инструментом для последовательной генерации: они хорошо моделируют дальние зависимости и естественно применяются к событийным представлениям музыки, что показано, например, в *Music Transformer*, *Pop Music Transformer* и последующих работах [??], [??], [??]. Их главный недостаток для генерации *MIDI* по тексту — высокая чувствительность к токенизации и отсутствие явной гарантии музыкально-теоретической согласованности.

Вариационные автокодировщики удобны для интерполяции и управления скрытым представлением, однако нередко уступают авторегрессионным моделям по локальной детализации и долгосрочной форме [??], [??]. Генеративно-состязательные сети сыграли заметную роль в многодорожечной генерации, но для дискретной символьной музыки остаются нестабильными и плохо согласуют глобальные музыкальные ограничения [??], [??]. Диффузионные модели показывают

высокое качество и позволяют добавлять управление правилами, но обладают большой вычислительной стоимостью и пока не снимают проблему целостной формы [??], [??], [??], [??]. Наконец, подходы на основе крупных языковых моделей привлекательны возможностью совместной работы с текстом и музыкой, но они переносят в музыкальную область и типичные проблемы моделей, дообученных по инструкциям: потерю предметной точности при слабой адаптации и деградацию общих способностей при слишком агрессивном дообучении [??], [??], [??], [??].

Таким образом, общий вывод обзора состоит в том, что ни одна архитектура сама по себе не обеспечивает одновременно хорошее текстово-музыкальное соответствие, теоретическую осмысленность и устойчивую структуру на длинных промежутках. Это и мотивирует переход к внешним критериям качества и подходам обучения с подкреплением [??], [??].

Метрики и современные решения генерации *MIDI* по тексту

Оценка генеративных музыкальных систем остаётся отдельной сложной задачей. Субъективные экспертные оценки хорошо отражают музыкальность результата, но плохо масштабируются. Объективные статистические метрики позволяют сравнивать распределения событий, однако часто слабо коррелируют с музыкальной выразительностью. Метрики на основе вложений и косвенные показатели для конкретной задачи частично закрывают этот разрыв, но не устраняют его полностью [??], [??], [??], [??]. Для генерации *MIDI* по тексту этого недостаточно вдвойне: необходимо оценивать и внутреннюю музыкальную правдоподобность, и согласование с текстом.

Содержательный переход к условной символьной генерации по тексту произошёл от работ по управлению эмоциями и атрибутами к системам, где текст напрямую связан с музыкальным выводом [??], [??]. Одним из наиболее прикладных решений стала модель *Text2midi*, формулирующая задачу как генерацию символьной музыки по текстовым описаниям [??]. В соседнем направлении *ChatMusician* трактует музыку как второй язык и показывает, что крупные языковые модели можно адаптировать для музыкального понимания и генерации [??]. Однако такие подходы по-разному балансируют между удобством текстового интерфейса и музыкальной строгостью: ABC-нотация облегчает перенос методов обработки естественного языка, но не совпадает с прикладной целью генерации *MIDI*; сквозная генерация *MIDI* по тексту ближе к практике, но сильнее зависит от качества текстовых подписей и сложности многорожечных примеров [??], [??].

Особенно важной для данной работы является статья *How Far Can Pretrained LLMs Go in Symbolic Music? Controlled Comparisons of Supervised and Preference-based Adaptation* [??]. Авторы сравнивают LLaMA 3.1 8B, дообученную по инструкциям, контролируемое дообучение (*SFT*), прямую оптимизацию предпочтений (*DPO*) и специализированную музыкальную базовую модель

на задачах генерации и понимания в ABC-нотации. Вариант обучения по предпочтениям строится не на явном критике, а на слабых предпочтениях: “отвергнутые” ответы получают путём музыкально мотивированных деградаций целевой ABC-записи при сохранении синтаксической корректности; после этого применяется *DPO*. Результаты работы показывают, что *SFT* обычно лучше адаптирует модель к музыкальному домену по метрикам на уровне токенов, тогда как *DPO* в большей степени сохраняет общие возможности модели, но не даёт устойчивого преимущества в музыкальном качестве. Авторы также специально подчёркивают, что перплексия и *Fréchet Music Distance* ведут себя по-разному и не дают единой картины качества [??]. Для настоящей работы эта статья важна в двух отношениях: она подтверждает полезность адаптации по предпочтениям в символьной музыке, но одновременно показывает, что слабого сигнала предпочтения недостаточно для уверенного улучшения структурного качества. Следовательно, вместо неявного сигнала предпочтения целесообразно рассмотреть более предметно ориентированный внешний критик.

Предлагаемый подход

В качестве базовой системы в работе рассматривается *Text2midi* как генератор *MIDI* по текстовому описанию [??]. Поверх него предлагается последующее дообучение с помощью *Group Relative Policy Optimization (GRPO)*, где политика получает награду за соответствие музыки текстовому запросу. Такой базовый вариант близок к современным подходам согласования модели с целевыми критериями: модель оптимизируется не только по вероятности обучающих последовательностей, но и по внешнему критерию, отражающему конечную задачу [??], [??].

Основная идея расширения состоит в добавлении графового критика. Последовательность *MIDI* преобразуется в музыкальный граф, в котором вершины и рёбра кодируют ноты, аккорды, моменты начала звучания, метрические позиции и связи между ними. Графовая нейронная сеть оценивает не токеновую правдоподобность, а структурные свойства результата: согласованность мелодии и гармонии, локальную функциональность аккордов, связь между событиями разных дорожек и более общую композиционную связность [??], [??], [??], [??]. Тогда итоговая награда сочетает как минимум два компонента: межмодальное согласование с текстом и структурную оценку графового критика.

Такой подход отличается и от чистого контролируемого дообучения (*SFT*), и от обучения по предпочтениям на деградированных примерах. При *SFT* модель приближает эмпирическое распределение токенов; при *DPO* по слабым предпочтениям она учится предпочитать один выход другому. В предлагаемой схеме вводится предметно-информированный источник сигнала, напрямую ориентированный на музыкальную структуру.

1 Данные и их предварительная обработка

В данной главе описывается используемый набор данных, обосновывается выбор корпуса *HookTheory Dataset* [??], а также приводится процедура его предварительной обработки и преобразования в графовое представление. Отдельно рассматриваются основные статистические свойства корпуса, поскольку распределение музыкально-теоретических признаков напрямую влияет на качество обучения графового критика.

1.1 Выбор набора данных

В качестве основного источника данных в работе был выбран *HookTheory Dataset*, компактная выгрузка данных из сервиса *HookTheory*, данный сервис предоставляет пользователям возможность аннотации музыкальных произведений с точки зрения голосоведения и аккомпанемента (см. Рисунок ??).

“Hey Jude” by The Beatles

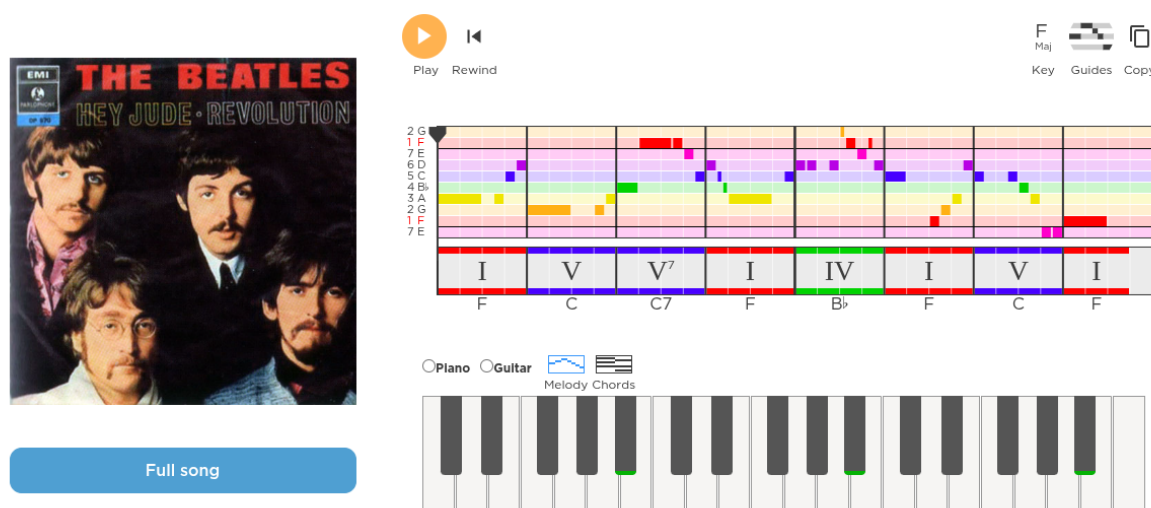


Рисунок 1 – Пример песни и её аннотации с сайта *HookTheory*

Данный корпус содержит музыкальные фрагменты в символьном представлении, включающие аннотированные партии мелодии и гармонического сопровождения. Такой формат особенно важен для данной работы, поскольку целевая задача состоит в оценке и последующем улучшении музыкально-теоретической согласованности *MIDI*-фрагментов, сгенерированных моделями генерации музыки по тексту.

В отличие от задачи генерации музыки напрямую, в данной работе требуется построить модель, способную анализировать уже имеющийся музыкальный фрагмент. Следовательно, существенным является не только наличие нотных событий, но и наличие информации о связи между ме-

лодий, аккордами, тональностью, ладом и метрическим положением событий. *HookTheory Dataset* близок к *lead-sheet*-представлению музыкального материала: фрагмент описывается через мелодическую линию, аккордовую последовательность и связанную с ними музыкально-теоретическую разметку.

Такое представление позволяет строить обучающие примеры, в которых можно анализировать согласованность мелодии с текущим аккордом, соответствие аккордовой последовательности тональному контексту, появление неаккордовых звуков, связь музыкальных событий с метрической сеткой, а также локальные и глобальные нарушения музыкально-теоретической структуры.

Существуют более крупные *MIDI*-корпуса, например *Lakh MIDI Dataset* [??] и *GiantMIDI-Piano* [?]. Они полезны для крупномасштабного предобучения моделей музыки в символьном представлении, однако не содержат достаточно подробной музыкально-теоретической разметки. В частности, в них отсутствует явная локальная связь между мелодическими событиями, аккордами, тональностью и гармонической функцией. Их использование потребовало бы отдельного этапа автоматического извлечения аккордов и тональностей, что внесло бы дополнительный шум в обучающие данные.

Другую группу составляют корпуса функционально-гармонического анализа, такие как *When in Rome* [?], *AugmentedNet* [?] и *Distant Listening Corpus* [?]. Эти корпуса содержат более строгую теоретическую разметку, однако в основном ориентированы на классическую нотную традицию и задачу гармонического анализа с римскими цифрами. В данной работе, напротив, требуется модель, применимая к *MIDI/lead-sheet*-представлению и к результатам генерации *MIDI* по текстовому описанию, которые часто ближе к популярной песенной музыке, чем к академической классической партитуре.

Также существуют крупные корпуса аккордовых последовательностей, например *ChoCo* [?] и *Chordonomicon* [?]. Они могут быть полезны для моделирования гармонических прогрессий, однако не всегда содержат мелодические партии и не позволяют напрямую оценивать соответствие конкретных нот *MIDI* гармоническому контексту. Для данной работы это является существенным ограничением, поскольку модель должна оценивать не только правдоподобие аккордовой последовательности, но и согласованность мелодии и аккомпанемента.

При этом *HookTheory Dataset* также имеет ограничения. Во-первых, в нём представлены не полные произведения, а короткие фрагменты, соответствующие отдельным секциям музыкальной формы: куплетам, припевам, вступлениям и т.д. Во-вторых, значительная часть разметки имеет пользовательское происхождение, что может приводить к отдельным ошибкам или неоднозначностям. В-третьих, корпус не является универсальным представлением всей западной музыкальной теории и преимущественно отражает практику *lead-sheet*-разметки популярной музыки.

Тем не менее для поставленной задачи эти ограничения не являются критическими. Цель работы состоит не в построении универсальной системы академического гармонического анализа, а в обучении модели, способной оценивать музыкально-теоретическую согласованность *MIDI*-фрагментов. В этом контексте *HookTheory Dataset* представляет собой наиболее релевантный компромисс: он содержит достаточно богатую теоретическую информацию, включает связь между мелодией и гармонией и остаётся близким к формату *MIDI/lead-sheet*, характерному для современных задач генерации музыки в символьном представлении.

1.2 Первичная обработка данных

Исходный корпус *HookTheory Dataset* хранится в формате *JSON*. Каждый музыкальный фрагмент представлен отдельным *JSON*-объектом, содержащим метаданные о треке, а также массивы мелодических и аккордовых событий. Поскольку не все поля исходного корпуса являются полезными для решаемой задачи, а часть признаков требует нормализации, обработка данных выполнялась в несколько этапов.

Общий процесс обработки включает: отбор информативных полей; нормализацию категориальных и числовых признаков; музыкально-теоретическую нормализацию тональностей, ладов, ступеней и аккордовых признаков; добавление специальных служебных полей; преобразование обработанных объектов в иерархические гетерогенные графы.

На первом этапе из исходного корпуса удаляются системные поля, не несущие музыкально-теоретической информации: ссылки на страницы песен в сервисе *HookTheory*, информация об авторах и аннотаторах, служебные идентификаторы и другие поля, не используемые в модели. После этого для каждого музыкального фрагмента сохраняются общая метаданная, массив мелодических событий, массив аккордовых событий и информация о секции музыкальной формы, если она представлена в данных.

Далее выполняется нормализация признаков. Тональности переводятся в частотные классы, то есть классы высот по модулю 12:

$$C/B^{\sharp} \rightarrow 0, \quad C^{\sharp}/D^{\flat} \rightarrow 1, \quad D \rightarrow 2, \quad \dots, \quad B/C^{\flat} \rightarrow 11.$$

Аналогичные словари используются для ладов, мелодических ступеней, типов аккордов, ступеней аккордов, обращений и дополнительных аккордовых признаков. Такое представление позволяет заменить строковые категории числовыми идентификаторами и унифицировать дальнейшую обработку данных.

Отдельная нормализация применяется к ладам. Каждый лад представляется как упорядо-

ченный массив смещений частотных классов относительно тоники:

$$\begin{aligned} \text{major} &= (0, 2, 4, 5, 7, 9, 11), & \text{minor} &= (0, 2, 3, 5, 7, 8, 10), \\ \text{dorian} &= (0, 2, 3, 5, 7, 9, 10), & \text{phrygian} &= (0, 1, 3, 5, 7, 8, 10), \\ \text{lydian} &= (0, 2, 4, 6, 7, 9, 11), & \text{mixolydian} &= (0, 2, 4, 5, 7, 9, 10), \\ \text{locrian} &= (0, 1, 3, 5, 6, 8, 10). \end{aligned}$$

Такой формат описывает лад как набор относительных ступеней, то есть как смещения в полутонах от тоники по модулю 12. Абсолютные частотные классы для конкретной тональности вычисляются по формуле:

$$pc_{\text{abs}} = (pc_{\text{tonic}} + pc_{\text{mode}}) \bmod 12,$$

где pc_{tonic} — частотный класс тоники, а pc_{mode} — смещение внутри ладового шаблона.

Например, для тональности *D major* имеем $pc_{\text{tonic}} = 2$. Тогда мажорный шаблон $(0, 2, 4, 5, 7, 9, 11)$ переходит в абсолютные частотные классы $(2, 4, 6, 7, 9, 11, 1)$. Такое представление удобно тем, что один и тот же ладовый шаблон может использоваться для любой тональности. Кроме того, оно позволяет выполнять музыкально-теоретические операции: проверять принадлежность ноты ладу, определять нетональные ноты, сравнивать основной лад и заимствованные, строить аккорды по терциям и формировать кандидаты для аккордового оценщика. После нормализации в словари категориальных признаков добавляются служебные поля: *PAD*, *MASK*, *UNK*, *NONE*. Они используются для параллельной обработки, маскирования признаков, обработки неизвестных значений и представления отсутствующих признаков.

На финальном этапе каждый музыкальный фрагмент преобразуется в иерархический гетерогенный граф. В графе используются следующие типы узлов: *song*, *bar*, *onset*, *note*, *chord*. Узел *song* соответствует всему музыкальному фрагменту и содержит глобальные признаки: тональность, лад, размер, *BPM* и длину фрагмента. Узел *bar* задаёт такт внутри метрической сетки. Узел *onset* соответствует временной позиции, в которой может начинаться одно или несколько событий. Узлы *note* и *chord* соответствуют мелодическим и аккордовым событиям. Между узлами строятся иерархические и последовательные рёбра:

$$\begin{aligned} \text{bar} &\rightarrow \text{next_bar}, \\ \text{onset} &\rightarrow \text{next_onset}, \\ \text{song} &\rightarrow \text{bar} \rightarrow \text{onset} \rightarrow \{\text{note}, \text{chord}\}, \quad \text{note} \rightarrow \text{next_note}, \\ \text{chord} &\rightarrow \text{covers_note}, \\ \text{chord} &\rightarrow \text{next_chord}. \end{aligned}$$

Таким образом, исходный *JSON*-объект *HookTheory* преобразуется в структурированный граф, пригодный для обучения гетерогенной графовой нейронной сети (см. Рисунок ??, Рисунок ??). Такое представление позволяет анализировать не только отдельные ноты и аккорды, но и отношения между ними, что является принципиально важным для построения музыкально-теоретического критика.

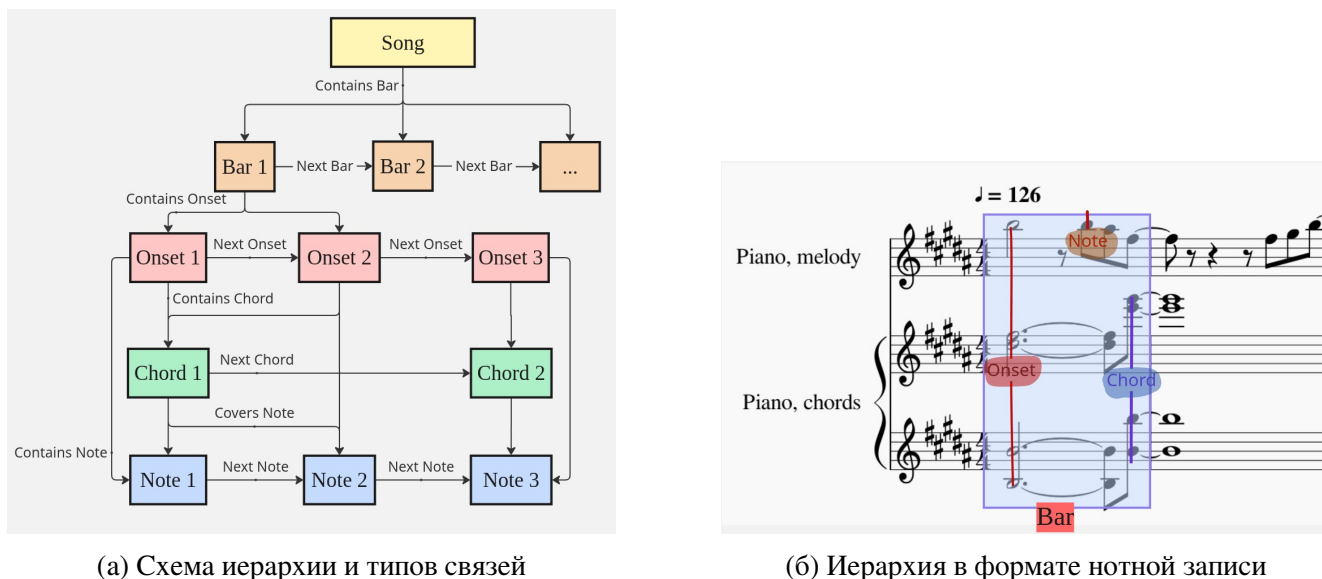


Рисунок 2 – Графовое представление:

1.3 Разведочный анализ данных

После первичной обработки был проведён разведочный анализ корпуса. В работе используется 26175 музыкальных фрагментов, разделённых на обучающую, валидационную и тестовую подвыборки в соотношении 80%/10%/10% соответственно. В общей сложности корпус содержит 449072 аккордовых события и 1338346 мелодических событий.

Основная ценность *HookTheory Dataset* состоит в том, что он содержит не только нотные события, но и метаинформацию и музыкально-теоретические аннотации. К общей метаинформации относятся тональность, лад, темп и ритмическая сетка. Аккордовые события содержат степень аккорда, тип аккорда, обращение, добавочные ступени, альтерации, задержания, признаки заимствования из другого лада, длительность и временную позицию. Мелодические события описываются степенью лада, октавным регистром, длительностью и временной позицией.

Анализ распределений показал, что данные имеют выраженный дисбаланс по ряду признаков. В распределении тональностей заметно доминирование тональности *C*, тогда как альтерированные тональности представлены существенно реже (см. ??). Распределение ладов также является неравномерным: наиболее распространённые лады представлены значительно лучше, чем редкие

модальные разновидности (см. ??). В рамках данной работы фрагменты в гармоническом миноре и фригийском доминантовом ладу были исключены из рассмотрения, поскольку число таких примеров недостаточно для устойчивого обучения и объективной оценки модели.

Для ритмической сетки наблюдается ожидаемое доминирование четырёхдольной пульсации. Данный дисбаланс не является критическим для поставленной задачи, поскольку основное внимание в работе уделяется не генерации ритма, а оценке мелодико-гармонической согласованности.

В аккордовых событиях заметно доминирование аккордов, построенных на первой ступени тональности (см. ??). Это соответствует музыкально-теоретическим ожиданиям, поскольку тоническая функция является одной из наиболее устойчивых и частотных в тональной музыке. По типам аккордов преобладают трезвучия и септаккорды, тогда как более сложные структуры представлены значительно реже. Распределение типов аккордов представлено отдельно (см. ??). Среди мелодических событий также наблюдается неравномерность: диатонические ступени встречаются существенно чаще, чем альтерированные, а один из октавных регистров доминирует над остальными (см. ??).

Наличие дисбаланса является важным ограничением корпуса. Однако в данной работе он не рассматривается как ошибка данных, поскольку во многом отражает реальные статистические закономерности западной популярной музыки. Для задачи оценки музыкально-теоретической согласованности важно, чтобы модель хорошо усваивала наиболее типичные и устойчивые отношения между мелодией, аккордами и тональностью. Именно такие отношения составляют основную часть корпуса.

При этом результаты модели следует интерпретировать прежде всего в рамках наиболее представленных областей набора данных: диатонической тональности, распространённых ладов, типичных аккордовых функций, трезвучий и септаккордов. Редкие классы могут рассматриваться как направление для дальнейшего расширения корпуса и улучшения модели.

Вывод по главе. В данной главе был обоснован выбор *HookTheory Dataset*, описан процесс его обработки и проведён разведочный анализ данных. Несмотря на наличие дисбаланса, корпус является подходящим для поставленной задачи, поскольку содержит связь между мелодией, аккордами, тональностью и метрической структурой. Это позволяет использовать его для построения графового критика, оценивающего музыкально-теоретическую согласованность *MIDI*-фрагментов.

2 Постановка задачи и архитектура графового критика

В данной главе формулируется задача построения графового критика — модели, оценивающей музыкально-теоретическую согласованность *MIDI*-фрагмента. Критик рассматривается как отдельный модуль оценки, который присваивает музыкальному фрагменту скалярную оценку. В дальнейшем такая оценка используется как награда при дообучении *text2midi*-модели.

Архитектура критика строится как система из двух моделей: графового учителя и графового ученика. Учитель является более информированной моделью: на вход он получает расширенное графовое представление музыкального примера, включающее структуру музыкального фрагмента, временную сетку, ноты, аккорды и дополнительные музыкально-теоретические признаки. Ученик является более компактной моделью, предназначенной для практического применения: он работает с *MIDI*-фрагментом и признаками, которые можно извлечь автоматически, после чего пытается воспроизвести оценки учителя (см. Рисунок ??).

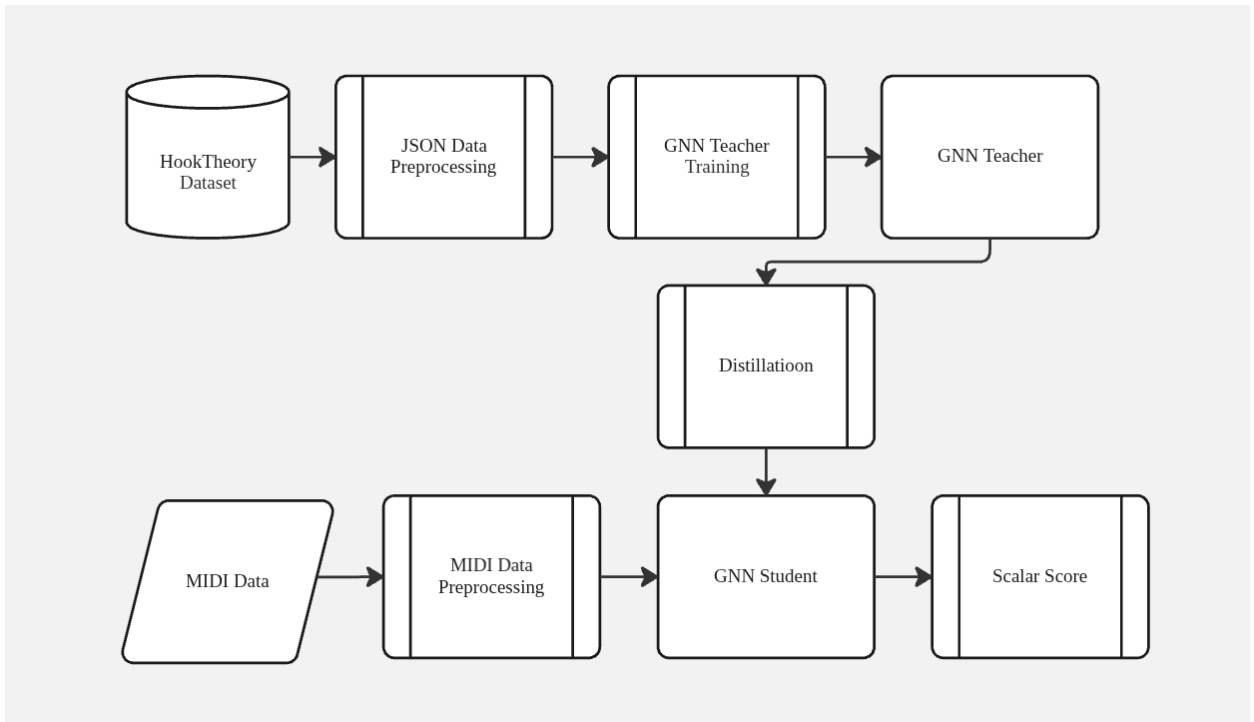


Рисунок 3 – Схема применения графового критика

Формально музыкальный пример представляется как гетерогенный граф:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, X),$$

где V — множество вершин, E — множество рёбер, τ_V задаёт тип вершины, τ_E задаёт тип ребра, а $X = \{x_v\}_{v \in V}$ — множество признаков вершин. Вершины соответствуют различным музыкальным объектам: фрагменту, тактам, временным позициям, нотам и аккордам. Рёбра

описывают как иерархические связи, например принадлежность ноты временной позиции, так и последовательные связи, например переход от одной ноты или временной позиции к следующей.

Цель критика состоит в построении функции

$$s_{\theta}(\mathcal{G}) \in \mathbb{R},$$

которая отображает музыкальный граф в скалярную оценку. Чем выше значение $s_{\theta}(\mathcal{G})$, тем более согласованным с музыкально-теоретической точки зрения считается фрагмент.

2.1 Графовый учитель

Графовый учитель представляет собой гетерогенную графовую нейронную сеть, обучаемую на задаче различения исходных музыкальных графов и их искусственно искажённых версий. Основная идея состоит в том, что корректный музыкальный пример должен получать более высокий *score*, чем пример, в котором нарушены локальные или глобальные музыкальные связи.

Пусть для исходного графа $\mathcal{G}$ строится искажённый граф $\tilde{\mathcal{G}}$. Тогда модель обучается выполнять условие:

$$s_{\theta}(\mathcal{G}) > s_{\theta}(\tilde{\mathcal{G}}).$$

Искажения применяются к музыкальным объектам графа. Они могут включать сдвиг ноты во времени, изменение аккорда, замену локального элемента, нарушение связи между мелодией и гармонией, а также другие теоретически обоснованные искажения. Такой подход не требует внешней ручной разметки качества: пары «корректный пример — испорченный пример» строятся автоматически, что позволяет обучать критик в *self-supervised* режиме.

Поскольку вершины разных типов имеют различные признаки, сначала признаки каждой вершины проецируются в общее скрытое пространство:

$$h_v^{(0)} = \phi_{\tau_v}(x_v),$$

где ϕ_{τ_v} — *encoder* для типа вершины τ_v , реализованный как *MLP*.

Далее применяется несколько слоёв гетерогенной *GNN*. Для каждого типа ребра r сообщения вычисляются отдельно:

$$m_{u \rightarrow v}^{(l,r)} = W_r^{(l)} h_u^{(l)}.$$

После этого сообщения от соседей агрегируются:

$$m_v^{(l)} = \sum_{r \in R} \text{AGG}_{u \in N_r(v)} m_{u \rightarrow v}^{(l,r)}.$$

Состояние вершины обновляется с *residual*-связью, нормализацией и нелинейностью:

$$h_v^{(l+1)} = \text{ReLU}(\text{LN}(h_v^{(l)} + m_v^{(l)})).$$

После нескольких *GNN*-слоёв модель получает контекстуализированные представления нот, аккордов, временных позиций и других узлов. Затем выполняется *pooling* по типам вершин:

$$g_t = \frac{1}{|V_t|} \sum_{v \in V_t} h_v^{(L)},$$

где V_t — множество вершин типа t .

После объединения векторов разных типов получается общий *embedding* графа:

$$g = \psi(g_1 \| g_2 \| \dots \| g_T),$$

где $\|$ обозначает конкатенацию, а ψ — линейная проекция или *MLP*. Итоговая оценка графа вычисляется *scoring*-головой:

$$s_\theta(\mathcal{G}) = f_{\text{score}}(g).$$

Для обучения используется *contrastive ranking loss*:

$$\mathcal{L}_{\text{rank}} = \text{softplus}\left(-\left(s_\theta(\mathcal{G}) - s_\theta(\tilde{\mathcal{G}}) - m\right)\right),$$

где m — *margin*. Эта функция штрафует модель, если *score* искажённого графа оказывается слишком близким к *score* исходного графа или превосходит его.

Дополнительно может использоваться бинарная классификационная компонента:

$$\mathcal{L}_{\text{bin}} = \frac{1}{2} \left(\text{BCE}(s_\theta(\mathcal{G}), 1) + \text{BCE}(s_\theta(\tilde{\mathcal{G}}), 0) \right).$$

Общая функция потерь учителя записывается в виде:

$$\mathcal{L}_{\text{teacher}} = \lambda_{\text{rank}} \mathcal{L}_{\text{rank}} + \lambda_{\text{bin}} \mathcal{L}_{\text{bin}} + \lambda_{\text{local}} \mathcal{L}_{\text{local}} + \lambda_{\text{rec}} \mathcal{L}_{\text{rec}}.$$

Здесь $\mathcal{L}_{\text{local}}$ отвечает за локальное обнаружение испорченных нот, аккордов или времен-

ных позиций, а $\mathcal{L}_{\text{rec}}$ используется при восстановлении замаскированных музыкальных признаков. Общая схема обучения представлена ниже (см. Рисунок ??).

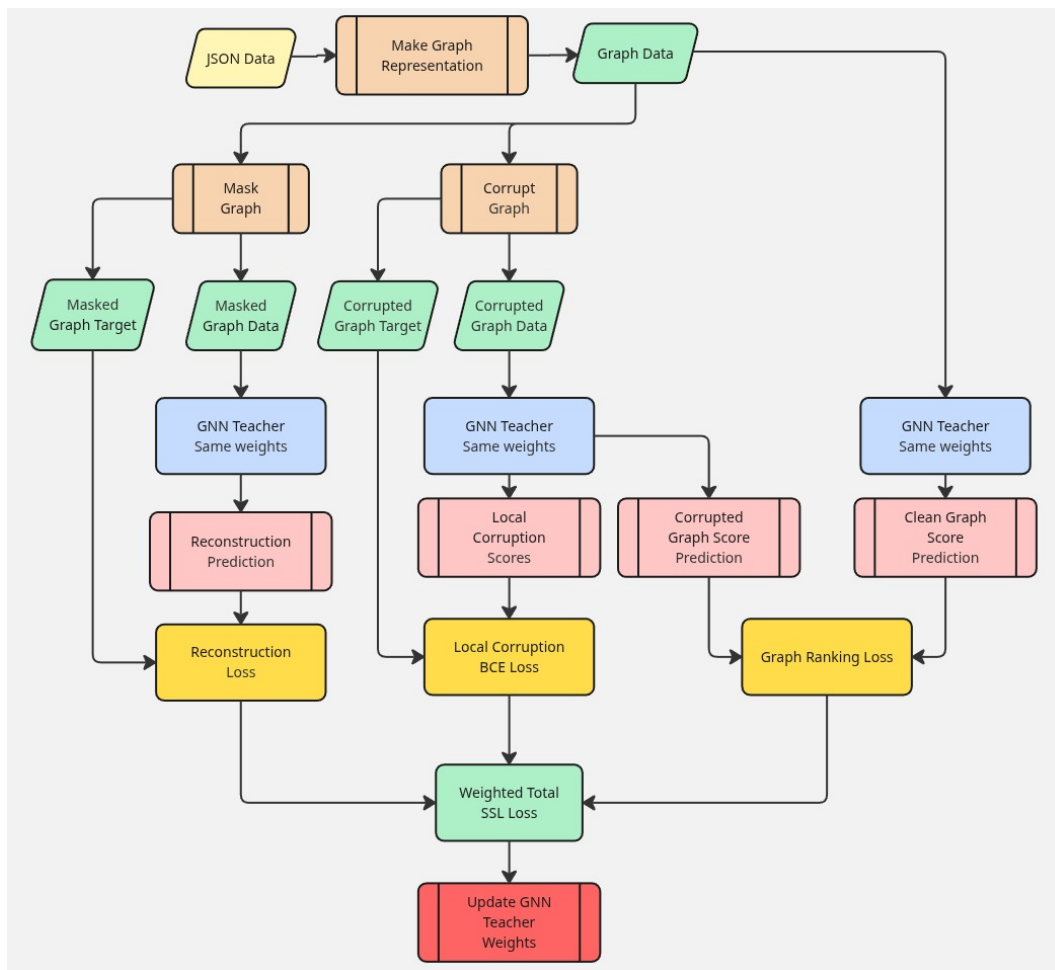


Рисунок 4 – Общая схема обучения графового учителя

2.2 Графовый ученик

Графовый ученик является уменьшенной и более прикладной версией графового учителя. Его основное отличие состоит в том, что он не получает расширенную информацию, доступную учителю, а работает только с *MIDI*-фрагментом и автоматически извлекаемыми признаками. Это делает ученика пригодным для использования в дальнейшем цикле генерации и оценки *MIDI*.

На вход ученику подаётся *MIDI*-фрагмент, из которого строится граф:

$$\mathcal{G}_M = (V_M, E_M, X_M).$$

Вершины такого графа соответствуют базовым объектам *MIDI*: фрагменту, тактам, временным позициям, нотам и предполагаемым аккордам. Рёбра задают временную и структурную

организацию: такты связаны с временными позициями, временные позиции — с нотами и аккордами, а соседние ноты и аккорды соединяются последовательными рёбрами.

Так как *MIDI* не всегда содержит готовую гармоническую разметку, для ученика важен отдельный этап извлечения аккордов. Аккорд восстанавливается по набору нот, звучащих одновременно или почти одновременно в небольшом временном окне w . Для этого из *MIDI*-событий строится множество частотных классов:

$$P_w = \{p_i \bmod 12 \mid p_i \in w\},$$

где p_i — *MIDI высота* ноты, а операция $\bmod 12$ переводит абсолютную высоту в класс высоты. Например, ноты C_4 , E_4 , G_4 , Bb_4 дают множество

$$P_w = \{C, E, G, Bb\}.$$

Далее перебирается множество кандидатов аккорда $c \in \mathcal{C}$. Каждый кандидат задаёт корень, тип аккорда, возможное обращение, набор основных тонов аккордового тела, а также дополнительные признаки: добавленные ступени, задержания, альтерации, пропуски и заимствованный ладовый контекст. Для кандидата строится множество частотных классов, которое он способен объяснить:

$$B(c) = \{\text{pitch classes, входящие в тело аккорда } c\}.$$

После этого вычисляется *score* соответствия между наблюдаемыми нотами и кандидатом:

$$q(c, P_w) = q_+(c, P_w) - q_-(c, P_w),$$

где q_+ — положительный вклад объяснённых признаков, а q_- — штрафы за несоответствия.

Положительная часть *score* может быть записана как

$$\begin{aligned} q_+(c, P_w) = & \lambda_1 \text{mode_match}(c) + \lambda_2 \text{bass_match}(c) + \lambda_3 |P_w \cap B(c)| \\ & + \lambda_4 \text{explained_extras}(c, P_w). \end{aligned}$$

Интерпретация компонент положительной части *score* приведена в Таблице ??.

Таблица 1 – Компоненты положительной части аккордового *score*

Компонента	Интерпретация
$\text{mode_match}(c)$	Поощряет кандидата, если его ладовый контекст совпадает с основным контекстом фрагмента.
$\text{bass_match}(c)$	Поощряет совпадение басовой ноты с ожидаемым басом аккорда или его обращением.
$ P_w \cap B(c) $	Показывает, сколько наблюдаемых <i>pitch classes</i> объясняется телом аккорда.
$\text{explained_extras}(c, P_w)$	Учитывает дополнительные ноты, которые не входят в простое трезвучие, но могут быть объяснены как септима, нона, задержание или альтерация.

Штрафная часть имеет вид

$$\begin{aligned}
 q_-(c, P_w) = & \mu_1 |\text{Unexplained}(c, P_w)| + \mu_2 |\text{MissingCore}(c, P_w)| \\
 & + \mu_3 \text{borrowed_mode_penalty}(c) + \mu_4 \text{mode_distance}(c) \\
 & + \mu_5 \text{body_size_penalty}(c) + \mu_6 \text{complexity_penalty}(c).
 \end{aligned}$$

Интерпретация компонент штрафной части *score* приведена в Таблице ??.

Таблица 2 – Компоненты штрафной части аккордового *score*

Компонента	Интерпретация
$\text{Unexplained}(c, P_w)$	Наблюдаемые <i>pitch classes</i> , которые кандидат не может объяснить.
$\text{MissingCore}(c, P_w)$	Обязательные тона аккорда, отсутствующие в <i>MIDI</i> -окне.
$\text{borrowed_mode_penalty}(c)$	Штрафует за использование заимствованного лада без достаточных оснований.
$\text{mode_distance}(c)$	Учитывает удалённость кандидата от текущего ладового контекста.
$\text{body_size_penalty}(c)$	Не даёт модели выбирать чрезмерно сложные аккорды без необходимости.
$\text{complexity_penalty}(c)$	Объединяет штрафы за добавленные ступени, задержания, альтерации и пропущенные обязательные тона.

Итоговый кандидат выбирается по максимуму:

$$\hat{c} = \arg \max_{c \in \mathcal{C}} q(c, P_w).$$

Например, пусть в момент времени t наблюдаются *MIDI*-ноты:

$$C - E - G - B\flat.$$

Тогда

$$P_t = \{C, E, G, B\flat\}.$$

Рассмотрим несколько кандидатов.

Кандидат 1: $C7$.

$$B(C7) = \{C, E, G, B\flat\}.$$

Этот кандидат объясняет все наблюдаемые *pitch classes*:

$$P_t \cap B(C7) = \{C, E, G, B\flat\}.$$

У него нет отсутствующих основных тонов и нет необъяснённых нот:

$$\text{MissingCore}(C7, P_t) = \emptyset, \quad \text{Unexplained}(C7, P_t) = \emptyset.$$

Поэтому *score* будет высоким: модель видит полный доминантсептаккорд без лишних нот.

Кандидат 2: C major.

$$B(C) = \{C, E, G\}.$$

Этот кандидат хорошо объясняет трезвучие C, E, G , но не объясняет $B\flat$:

$$\text{Unexplained}(C, P_t) = \{B\flat\}.$$

Если $B\flat$ не интерпретируется как допустимая добавленная или альтерированная ступень, кандидат получает штраф за лишнюю необъяснённую ноту. Поэтому его *score* ниже, чем у $C7$.

Кандидат 3: $Am7\flat5/C$.

Такой кандидат может иметь тело

$$B(Am7\flat5) = \{A, C, E\flat, G\}.$$

В наблюдаемых нотах есть C и G , но отсутствуют важные для этого кандидата A и $E\flat$, а

вместо $E\flat$ наблюдается E . Тогда возникают одновременно пропущенные основные тона и несоответствующие высотные классы:

$$\text{MissingCore}(Am7\flat5/C, P_t) = \{A, E\flat\},$$

а часть наблюдаемых нот плохо согласуется с телом кандидата. Несмотря на возможное совпадение баса C , общий *score* оказывается ниже, чем у $C7$.

Таким образом, алгоритм выбирает не просто самый большой аккорд и не просто ближайшее трезвучие, а кандидата, который лучше всего объясняет наблюдаемые *MIDI*-ноты при минимальном числе лишних допущений. В приведённом примере итоговое предсказание будет

$$\hat{c} = C7.$$

Такая оценка важна для графового ученика, потому что предсказанный аккорд затем превращается в вершину графа и влияет на признаки гармонического контекста, связи между нотами и аккордами, а также на итоговую оценку *MIDI*-фрагмента.

После построения *MIDI*-графа ученик использует компактную гетерогенную *GNN*. Для каждой вершины строится начальное представление:

$$z_v^{(0)} = \phi_{\tau_v}(x_v),$$

после чего несколько графовых слоёв обновляют его через сообщения от соседей:

$$z_v^{(l+1)} = \sigma \left(\text{LN} \left(z_v^{(l)} + \sum_r \text{AGG}_{u \in N_r(v)} W_r^{(l)} z_u^{(l)} \right) \right).$$

Далее применяется *pooling* по вершинам и строится *embedding MIDI*-графа:

$$z_G = \text{Pool}(\{z_v^{(L)}\}_{v \in V_M}).$$

Итоговая оценка ученика равна:

$$s_\varphi(\mathcal{G}_M) = f_{\text{student}}(z_G).$$

Ученик обучается не на ручных метках, а на оценках учителя. Если учитель присваивает музыкальному фрагменту *score* $s_\theta(\mathcal{G})$, то ученик минимизирует ошибку дистилляции:

$$\mathcal{L}_{\text{distill}} = |s_\varphi(\mathcal{G}_M) - s_\theta(\mathcal{G})|,$$

или её сглаженный вариант:

$$\mathcal{L}_{\text{distill}} = \text{SmoothL1}(s_{\varphi}(\mathcal{G}_M), s_{\theta}(\mathcal{G})) .$$

Для пар *clean/corrupted* дополнительно используется *ranking loss*:

$$\mathcal{L}_{\text{student-rank}} = \text{softplus}\left(-\left(s_{\varphi}(\mathcal{G}_M) - s_{\varphi}(\tilde{\mathcal{G}}_M)\right)\right) .$$

Эта компонента нужна для того, чтобы ученик не только приближал абсолютные оценки учителя, но и сохранял порядок предпочтений между корректными и искажёнными музыкальными примерами. Ниже представлена общая схема обучения графового ученика (см. Рисунок ??).

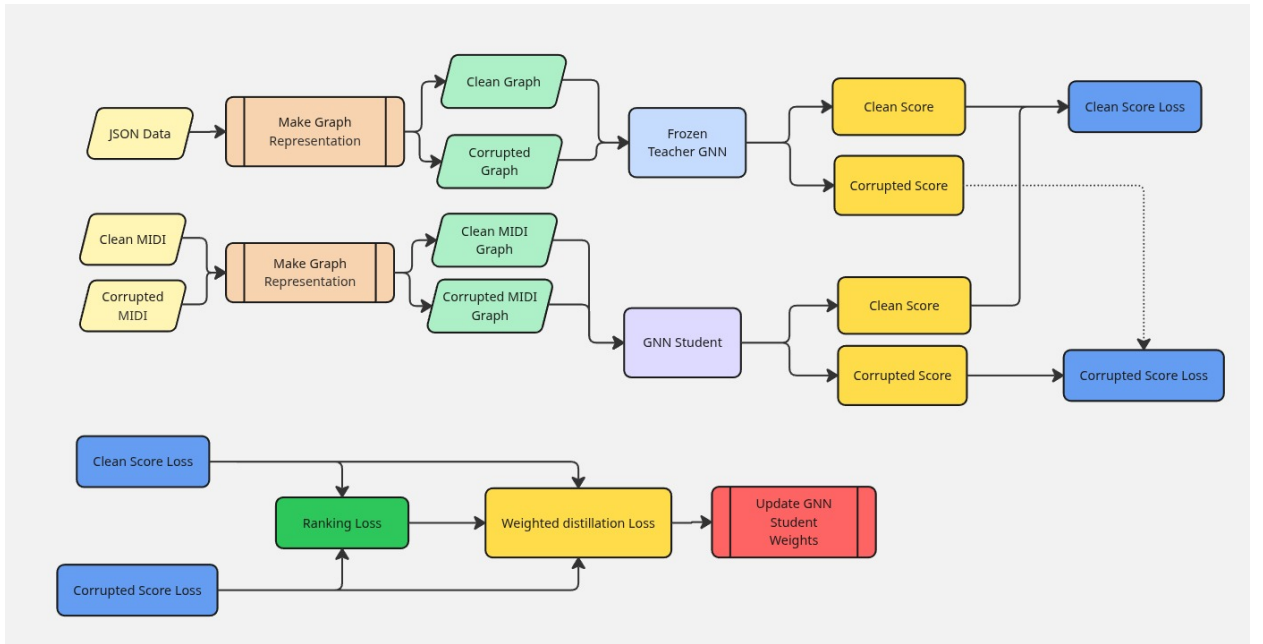


Рисунок 5 – Общая схема обучения графового ученика

2.3 Эксперименты графового учителя

Перед описанием отдельных экспериментов зафиксируем общую схему оценки. Экспериментальная часть состоит из двух блоков: сначала проверяется графовый учитель, который получает расширенный музыкально-теоретический граф, затем оценивается графовый ученик, работающий только с *MIDI*-представлением. В таблицах используются метрики качества ранжирования, восстановления аккордов и дистилляции *score*-функции.

Для оценки учительской-модели использовались следующие базовые *ranking*-метрики:

$$\text{IntraAcc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\left[s(\mathcal{G}_i) > s(\tilde{\mathcal{G}}_i)\right] ,$$

$$\text{InterAcc} = \frac{1}{N(N-1)} \sum_{i \neq j} \mathbf{1} \left[s(\mathcal{G}_i) > s(\tilde{\mathcal{G}}_j) \right],$$

$$\text{RankAcc} = \frac{1}{N^2} \sum_{i,j} \mathbf{1} \left[s(\mathcal{G}_i) > s(\tilde{\mathcal{G}}_j) \right].$$

Здесь $\mathcal{G}_i$ — исходный граф, $\tilde{\mathcal{G}}_i$ — его искажённая версия, а $s(\cdot)$ — *score* графового критика. Метрика *IntraAcc* показывает долю правильно упорядоченных пар *clean/corrupted* внутри одного примера, *InterAcc* сравнивает исходные графы с искажениями других примеров, а *RankAcc* объединяет все такие сравнения внутри оцениваемого набора. Префикс *Best* означает лучшее значение за время обучения.

Остальные метрики и служебные показатели, используемые в таблицах экспериментов, имеют следующую интерпретацию:

- *BinaryAcc* — точность бинарного различения исходных и искажённых графов после порогового решения по *score* или логиту модели.
- *SuccessRate* — доля случаев, где исходный граф получает более высокий *score*, чем его *OOD*-преобразование; *Mean gap* — средняя разность $s(\mathcal{G}) - s(\mathcal{G}')$; *Global RankAcc* — ранжировочная точность на всём наборе *OOD*-пар.
- *Applied samples* — число примеров, к которым удалось применить искажение или преобразование; *Applied rate* — соответствующая доля от проверяемого набора.
- *Top-1 exact* — доля случаев полного совпадения предсказанного аккорда с разметкой; *Top-5 contains GT* — доля случаев, где правильный аккорд входит в пять лучших кандидатов.
- *Root acc*, *Type acc*, *Root+Type acc*, *Mode acc* и *Inversion acc* оценивают соответственно правильность корня аккорда, типа аккорда, их совместного совпадения, ладового контекста и обращения.
- *Manual weights* и *Learned weights* в таблице по типам аккордов показывают точность восстановления внутри конкретного класса аккордов при ручных и обучаемых весах *scoring*-функции.
- *Pair rank acc* — доля корректно упорядоченных пар *clean/corrupted* для *student/observer*; *Batch rank acc* — доля корректных сравнений между всеми положительными и отрицательными примерами внутри батча.

- *Spearman* — ранговая корреляция между оценками *student* и *teacher*; *Pearson* — линейная корреляция между этими оценками.
- *MAE* — средняя абсолютная ошибка между *score* ученика и учителя; *Score distill loss* — значение функции потерь дистилляции для скалярного *score*. Для метрик со стрелкой $\uparrow$ лучше большее значение, а для метрик со стрелкой $\downarrow$ — меньшее.

Эксперименты графового учителя направлены на проверку того, способен ли *teacher* различать исходные и искажённые музыкальные графы, а также какие архитектурные и обучающие модификации повышают качество ранжирования.

2.3.1 *Smoke*-тесты и ранние *one-batch ablation*

Первый этап экспериментов представлял собой *smoke*-тесты и ранние *one-batch ablation*. Его цель состояла не в финальной оценке качества, а в проверке работоспособности *contrastive*-пайплайна. Для каждого исходного графа строилась искажённая версия, после чего модель обучалась выполнять условие $s_{\theta}(\mathcal{G}) > s_{\theta}(\tilde{\mathcal{G}})$. На этом этапе сравнивались простые *graph corruptions* и более содержательные *theory-aware corruptions*. Результаты приведены в Таблице ??.

Таблица 3 – *Smoke*-тесты и ранние *one-batch ablation*

<i>Corruption family</i>	<i>Pooling</i>	<i>Hybrid scorer</i>	<i>Epochs</i>	<i>Best rank acc</i> $\uparrow$
<i>Theory-aware ablation</i>	<i>mean_max</i>	<i>true</i>	500	1.0000
<i>Theory-aware ablation</i>	<i>mean_max</i>	<i>false</i>	500	1.0000
<i>Theory-aware ablation</i>	<i>mean</i>	<i>true</i>	500	1.0000
<i>Theory-aware ablation</i>	<i>mean</i>	<i>false</i>	500	1.0000
<i>Graph ablation</i>	<i>mean_max</i>	<i>true</i>	500	0.8750
<i>Graph ablation</i>	<i>mean</i>	<i>true</i>	500	0.8750
<i>Graph ablation</i>	<i>mean</i>	<i>false</i>	500	0.8750
<i>Graph ablation</i>	<i>mean_max</i>	<i>false</i>	500	0.6875

Результаты показали, что на раннем *sanity-check* уровне *theory-aware corruptions* дают более устойчивое поведение: лучшая метрика *rank acc* достигала 1.0000 во всех конфигурациях, тогда как простые *graph corruptions* давали менее стабильный результат. Это стало основанием использовать музыкально осмысленные искажения в дальнейших экспериментах.

2.3.2 Обобщение на *OOD-corruptions*

Дополнительно была проверена обобщающая способность графового критика на искажениях, которые не совпадают напрямую с основным набором обучающих искажений. Цель эксперимента состояла в проверке двух гипотез. Во-первых, если новое искажение действительно ухудшает музыкальный фрагмент, то учительская-модель должна понижать его *score* даже без прямого обучения на таком типе ошибки. Во-вторых, если преобразование является условно безвредным, то оно не должно превращаться для модели в ложный отрицательный сигнал.

Для исходного графа $\mathcal{G}_i$ и его преобразованной версии $\mathcal{G}'_i$ рассматривалась разность *score*:

$$\Delta_i = s(\mathcal{G}_i) - s(\mathcal{G}'_i).$$

Если $\Delta_i > 0$, то модель оценивает исходный граф выше преобразованного. Для вредных *OOD-corruptions* это считается правильной реакцией, а для безвредных преобразований систематически положительный *gap*, наоборот, был бы признаком лишнего шума. Основная метрика записывается как

$$\text{SuccessRate} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[s(\mathcal{G}_i) > s(\mathcal{G}'_i)].$$

Также оценивался средний *gap*:

$$\overline{\Delta} = \frac{1}{N} \sum_{i=1}^N (s(\mathcal{G}_i) - s(\mathcal{G}'_i)).$$

Результаты приведены в Таблице ??.

Таблица 4 – Проверка обобщения *teacher*-модели на *OOD-corruptions*

<i>Group</i>		<i>Applied samples</i>	<i>SuccessRate</i>	<i>Mean gap</i>	<i>Global RankAcc</i>	<i>Interpretation</i>
<i>Harmful corruptions</i>	<i>OOD</i>	4265	0.6080	0.6649	0.5504	Новые вредные искажения в среднем понижают <i>score</i>
<i>Benign transformations</i>		1763	0.5230	0.0110	0.5012	Безвредные преобразования почти не создают систематический штраф

Отдельные типы вредных *OOD*-искажений давали особенно выраженный сигнал. Например, *strong_weak_beat_flip* достиг *SuccessRate* = 0.7859 и среднего *gap* = 4.9666, то есть модель хорошо распознаёт нарушение метрически сильных и слабых долей. Удаление ноты из *onset* также давало устойчивый эффект: *SuccessRate* = 0.7594. Удаление аккорда из *onset* имело *SuccessRate* = 0.7282 и средний *gap* = 0.4860.

При этом более тонкие искажения, например локальный сдвиг фрагмента на полутон, нарушение октавного скачка или изменение длительности ноты, давали более слабый сигнал: их *SuccessRate* находился ближе к 0.5. Это показывает, что критик лучше обобщает на структурно и гармонически выраженные ошибки, чем на очень локальные или неоднозначные изменения.

Таблица 5 – Примеры вредных *OOD-corruptions*

<i>OOD corruption</i>	<i>Applied rate</i>	<i>SuccessRate</i> ↑	<i>Mean gap</i> ↑
strong_weak_beat_flip	0.906	0.7859	4.9666
drop_note_from_onset	0.906	0.7594	0.0096
drop_chord_from_onset	0.986	0.7282	0.4860
out_of_key_note	0.906	0.5850	0.0507
semitone_from_bass_or_chord_tone	0.902	0.5676	0.0610

Отдельно важен результат на безвредных преобразованиях. Для группы *benign transformations* средний *gap* оказался практически нулевым: 0.0110, а *Global RankAcc* = 0.5012, то есть почти совпал со случайным уровнем. Это означает, что *teacher*-модель не начинает автоматически штрафовать любое изменение *MIDI*-графа. Например, транспозиция с соответствующим изменением тоники дала *SuccessRate* = 0.4580 и средний *gap*, близкий к 0.0000, что соответствует ожидаемому поведению: музыкально эквивалентное преобразование не должно восприниматься как ошибка.

Таким образом, эксперимент показывает, что механизм искажающего-обучения не сводится к запоминанию конкретных типов обучающих искажений. Финальный учитель частично обобщает *learned scoring* на новые вредные нарушения, но при этом не превращает нейтральные музыкальные преобразования в дополнительный шумный отрицательный сигнал.

2.3.3 Влияние локального *attention*-контекста

Следующий этап был направлен на проверку влияния локального *attention*-контекста. Для локального контекста сравнивались *mean*-агрегация и *attention*-механизм. Результаты приведены в Таблице ??.

Таблица 6 – *Attention/local context ablation*

<i>Pooling</i>	<i>Local context</i>	<i>Hidden / layers</i>	<i>MLM / corr epochs</i>	<i>RankAcc</i> ↑	<i>IntraAcc</i> ↑	<i>InterAcc</i> ↑	<i>BinaryAcc</i> ↑
<i>mean_max</i>	<i>mean</i>	128 / 3	3 / 7	0.8081	0.8134	0.8078	0.7400
<i>mean_max</i>	<i>attention</i>	128 / 3	3 / 7	0.8133	0.8293	0.8125	0.7407
<i>attention</i>	<i>mean</i>	128 / 3	3 / 7	0.8047	0.7704	0.8058	0.7301
<i>attention</i>	<i>attention</i>	128 / 3	3 / 7	0.8076	0.7763	0.8087	0.7425

Лучшей конфигурацией по основным *ranking*-метрикам стала комбинация *mean_max pooling* и *attention local context*. Это показывает, что *attention* оказывается полезен именно в

локальном контексте, где модель должна определить, какие соседние ноты, аккорды и *onset*-узлы важны для обнаружения музыкального нарушения.

2.3.4 Поэтапное обучение

Для более честного сравнения в последующих экспериментах был использован фиксированный набор *corruption*-пар:

$$D_{\text{pairs}} = \{(\mathcal{G}_i, \tilde{\mathcal{G}}_i)\}_{i=1}^N.$$

Это позволило сравнивать разные режимы обучения на одном и том же наборе отрицательных примеров.

Далее проверялось влияние поэтапного обучения. Обучение делилось на две стадии. На первой стадии модель восстанавливала замаскированные признаки нот и аккордов, а на второй стадии включалось *contrastive*-обучение на *corruption*-парах. Сравнивались обучение *corruption-stage* с нуля и *staged*-вариант, в котором модель сначала проходила *MLM*-предобучение, а затем дообучалась на *corruption*-парах. Результаты представлены в Таблице ??.

Таблица 7 – *Staged training*

Training mode	Init	Hidden / layers	Pooling	Local context	Corr epochs	Best RankAcc ↑	Best IntraAcc ↑	Best InterAcc ↑	BinaryAcc ↑
<i>MLM</i> → <i>corruption</i>	<i>MLM checkpoint</i>	128 / 3	<i>mean_max</i>	<i>attention</i>	84	0.8998	0.9118	0.8994	0.7486
<i>Corruption from scratch</i>	<i>random</i>	128 / 3	<i>mean_max</i>	<i>attention</i>	84	0.8963	0.9007	0.8962	0.7489

MLM-предобучение дало небольшой выигрыш по лучшим *ranking*-метрикам: *Best RankAcc* вырос с 0.8963 до 0.8998, *Best IntraAcc* — с 0.9007 до 0.9118, а *Best InterAcc* — с 0.8962 до 0.8994. При этом обучение с нуля оказалось сопоставимым и немного лучше по *BinaryAcc*. Поэтому преимущество *MLM*-предобучения следует интерпретировать осторожно: оно полезно как способ стабилизации обучения, но не является универсально лучшим по всем метрикам.

2.3.5 Архитектурные модификации

Далее были проверены архитектурные модификации: *HGT-backbone*, динамическое взвешивание *loss*-компонент и *learned logit fusion*. Динамическое взвешивание задавалось через обучаемые параметры неопределённости:

$$\mathcal{L}_{\text{dyn}} = \sum_i (e^{-s_i} \mathcal{L}_i + s_i),$$

где s_i — обучаемый параметр для i -й компоненты функции потерь. *Learned logit fusion* объединял глобальный *score* графа и локальные *summary*-признаки:

$$s_{\text{final}} = f_{\text{fusion}}(s_{\text{global}} || r_{\text{local}}).$$

Результаты *ablation* приведены в Таблице ??.

Таблица 8 – *Feature ablation: dynamic weighting, HGT и learned logit fusion*

Variant	Backbone	Dynamic weights	Logit fusion	Hidden / layers	MLM / corr epochs	Best RankAcc ↑	Best IntraAcc ↑	Best InterAcc ↑	BinaryAcc ↑
Baseline	SAGE	no	no	128 / 3	20 / 40	0.8476	0.8547	0.8472	0.7668
HGT	HGT	no	no	128 / 3	20 / 40	0.8409	0.8410	0.8409	0.7537
Dynamic weights	SAGE	yes	no	128 / 3	20 / 40	0.8495	0.8530	0.8494	0.7731
Learned logit fusion	SAGE	no	yes	128 / 3	20 / 40	0.8519	0.8408	0.8523	0.7761

Эксперименты показали, что *HGT-backbone* не улучшил качество относительно *SAGE-backbone*. Наиболее полезными оказались *dynamic weighting* и *learned logit fusion*. *Dynamic weighting* улучшил баланс между компонентами функции потерь, а *learned logit fusion* дал более устойчивое качество по *ranking*- и *binary*-метрикам.

2.3.6 Финальная *teacher*-конфигурация

Финальная *teacher*-конфигурация объединяла наиболее удачные решения: *SAGE-backbone*, увеличенную скрытую размерность, четыре *GNN*-слоя, *local attention context*, *dynamic loss weighting* и *learned fusion*.

Таблица 9 – Финальная *teacher*-конфигурация

Teacher variant	Backbone	Dynamic weights	Logit fusion	Hidden / layers	Pooling	Local context	MLM / corr epochs	Best RankAcc ↑	Best IntraAcc ↑	Best InterAcc ↑	BinaryAcc ↑
Final local-theory dynfusion	SAGE	yes	yes	256 / 4	mean_max	attention	30 / 39	0.9249	0.9275	0.9250	0.8456

Финальный *local-theory dynfusion teacher* показал наиболее сильные результаты по всем основным *ranking*-метрикам. Модель достигла $Best\ RankAcc = 0.9249$, $Best\ IntraAcc = 0.9275$ и $Best\ InterAcc = 0.9250$. Это означает, что *teacher* корректно ранжирует не только *clean/corrupted*-пары одного примера, но и сохраняет преимущество *clean*-графов при сравнении с *corrupted*-графами других примеров внутри *batch*. Дополнительно $BinaryAcc = 0.8456$ показывает, что модель научилась отделять *clean* и *corrupted* графы не только относительно, но и в абсолютной шкале *score*.

2.4 Эксперименты графового ученика

Эксперименты графового ученика проверяют, насколько эффективно *observer/student*-модель восстанавливает признаки из *MIDI* и воспроизводит оценки финального *teacher*. В отличие от учителя, ученик не получает готовый расширенный граф, поэтому качество автоматического извлечения аккордов и дистилляции напрямую влияет на итоговую пригодность модели как *reward*-критика для *MIDI*-фрагментов.

2.4.1 Качество предсказания аккордов из *MIDI*

Так как *observer*-модель получает на вход только *MIDI*, важным этапом является восстановление аккордовых признаков. Для каждого *harmonic onset* строится множество звучащих *pitch classes*:

$$P_t = \{p_i \bmod 12 \mid \text{note}_i \text{ звучит в момент } t\}.$$

Для каждого кандидата аккорда c вычисляется *score*:

$$q(c, P_t) = w_{\text{match}} \text{match}(c, P_t) - w_{\text{miss}} \text{missing}(c, P_t) \\ - w_{\text{extra}} \text{extra}(c, P_t) - w_{\text{complex}} \text{complexity}(c).$$

В ручном варианте веса задавались эвристически. В *learned*-варианте веса подбирались по данным, после чего кандидат выбирался как

$$\hat{c}_t = \arg \max_c q(c, P_t).$$

Эксперимент `manual_vs_learned_fast_probe` сравнивал ручной *scoring* и *learned scoring* на задаче восстановления аккордов. Результаты приведены в Таблице ??.

Таблица 10 – Сравнение ручного и обучаемого *scoring* для восстановления аккордов из *MIDI*

<i>Scoring</i>	<i>Top-1 exact</i> ↑	<i>Top-5 contains GT</i> ↑	<i>Root acc</i> ↑	<i>Type acc</i> ↑	<i>Root+Type acc</i> ↑	<i>Mode acc</i> ↑	<i>Inversion acc</i> ↑
<i>Learned weights</i>	0.9216	0.9907	0.9753	0.9864	0.9690	0.9324	0.9811
<i>Manual</i>	0.7641	0.9184	0.9249	0.8368	0.8147	0.9061	0.9213

Обученные веса существенно улучшили восстановление аккордов. Наиболее заметный прирост наблюдается для типа аккорда: *Type acc* вырос с 0.8368 до 0.9864. Также сильно выросла точность полного совпадения: *Top-1 exact* увеличился с 0.7641 до 0.9216.

Дополнительно был проведён анализ качества предсказания в зависимости от типа аккорда. Результаты представлены в Таблице ??.

Таблица 11 – Качество восстановления аккордов по типам

<i>Type</i>	<i>Manual weights</i> ↑	<i>Learned weights</i> ↑
<i>triad</i> / 5	0.915	0.938
<i>seventh</i> / 7	0.227	0.867
<i>ninth</i> / 9	0.000	0.850
<i>eleventh</i> / 11	0.000	0.886
<i>thirteenth</i> / 13	0.000	0.089

Из таблицы видно, что эвристическая модель почти не угадывала сложные аккорды. Для *ninth*, *eleventh* и *thirteenth* точность ручного *scoring* равна нулю. После обучения весов качество для *ninth* и *eleventh* выросло до 0.850 и 0.886 соответственно. Для *thirteenth* качество осталось низким, что может быть связано с малым числом таких примеров и высокой неоднозначностью их *pitch-class* состава.

Этот результат важен для *observer*, потому что ошибка в типе аккорда меняет гармоническую структуру *MIDI*-графа. Более точное восстановление аккордов делает входной граф ученика ближе к графу, на котором обучался *teacher*, и уменьшает разрыв между *teacher*-моделью и *student*-моделью.

2.4.2 Дистилляция промежуточных представлений *teacher*

Следующий эксперимент проверял, помогает ли *observer* получать не только итоговый *scalar score* *teacher*-модели, но и промежуточные представления. В базовой постановке *student* минимизирует ошибку *score*-дистилляции и *ranking loss*. В расширенной постановке добавляются *loss*-компоненты для *graph embedding*, *pooled node-type embeddings* и *local summaries* *teacher*-модели:

$$\mathcal{L} = \lambda_s \mathcal{L}_{score} + \lambda_r \mathcal{L}_{rank} + \lambda_m \mathcal{L}_{margin} + \lambda_g \mathcal{L}_{graph} + \lambda_t \mathcal{L}_{type} + \lambda_l \mathcal{L}_{local}.$$

Здесь

$$\mathcal{L}_{graph} = \|P_g z_G^S - z_G^T\|_2^2,$$

$$\mathcal{L}_{type} = \sum_{\tau} \|P_{\tau} z_{\tau}^S - z_{\tau}^T\|_2^2,$$

$$\mathcal{L}_{local} = \|P_l u^S - u^T\|_2^2.$$

z_G^S и z_G^T — графовые *embeddings* *student*- и *teacher*-моделей, z_{τ}^S и z_{τ}^T — *pooled*-представления

по типам вершин, u^S и u^T — локальные *summary*-признаки. Матрицы P_g , P_τ , P_l являются *projection adapters*, позволяющими сопоставить пространства *student* и *teacher*.

Контролируемый эксперимент был проведён на одинаковом *subset 2k*. Варианты отличались только весами *intermediate loss*: в *no intermediate* они были равны нулю, а в *intermediate* использовались значения 0.1. Результаты приведены в Таблице ??.

Таблица 12 – Влияние дистилляции промежуточных представлений *teacher*-модели

Variant	Pair rank acc ↑	Batch rank acc ↑	Spearman ↑	Pearson ↑	MAE ↓	Score distill loss ↓
Intermediate	0.6797	0.6787	0.4173	0.3588	2.3121	3.7560
No intermediate	0.6970	0.6111	0.2712	0.1728	2.5432	4.2046

No-intermediate вариант достиг более высокого раннего пика по *Pair rank acc*, однако по более устойчивым метрикам качества дистилляции *intermediate*-подход оказался лучше. Он дал более высокий *Batch rank acc*, более высокие корреляции Спирмена и Пирсона, а также меньшие *MAE* и *score distillation loss*. Это показывает, что промежуточные представления учительской модели помогают *observer* не только запоминать порядок внутри отдельных пар, но и лучше приближать общую *score*-функцию учителя.

2.4.3 Финальное сравнение *observer*-конфигураций

В финальном сравнении сопоставлялись предыдущий *observer* без *intermediate distillation* и *observer*, обученный с промежуточными *teacher*-представлениями на финальном *teacher*. Это сравнение отражает уже не чистую абляцию, а практический итоговый пайплайн.

Таблица 13 – Финальное сравнение *observer*-конфигураций

Observer variant	Teacher / cache	Intermediate losses	Epochs	Pair rank acc ↑	Batch rank acc ↑	Spearman ↑	Pearson ↑	MAE ↓
Intermediate	final_teacher_mlm30_corr80	yes	20	0.8692	0.8464	0.6926	0.6873	2.0698
No intermediate	final_teacher_mlm30_corr80	no	20	0.8509	0.8387	0.7135	0.6832	1.4717

Intermediate observer показал лучшие *ranking*-метрики: *Pair rank acc* вырос с 0.8509 до 0.8692, а *Batch rank acc* — с 0.8387 до 0.8464. Это означает, что модель лучше воспроизводит относительные предпочтения учительской модели между *clean* и *corrupted MIDI*-графами.

При этом *no-intermediate* вариант сохранил меньшую абсолютную ошибку *MAE* и немного более высокий *Spearman*. Поэтому финальный результат можно интерпретировать так: *intermediate distillation* и новый учитель улучшают именно *ranking*-поведение ученика, что наиболее важно для *reward*-модели, но абсолютная калибровка *score* остаётся отдельной задачей для последующей настройки.

Вывод по главе. В данной главе была сформулирована задача графового критика и описана архитектура системы из двух моделей: графового учителя и графового ученика. Учитель обучается различать корректные и искажённые музыкальные графы, а ученик обучается воспроизводить оценки учителя по *MIDI*-представлению. Проведённые эксперименты показали, что *theory-aware corruptions*, локальный *attention*-контекст, динамическое взвешивание функции потерь и *learned fusion* повышают устойчивость учительской модели, а обучаемый аккордовый *scoring* и дистилляция промежуточных представлений улучшают практическое *ranking*-поведение ученической модели.

3 Постановка задачи text-to-midi

Задача text-to-midi состоит в генерации MIDI-последовательности по текстовому описанию музыкального фрагмента. Модель получает запрос на естественном языке и должна построить символическое музыкальное представление: ноты, длительности, ритмическую организацию, темп, инструменты и другие свойства композиции [??], [??], [??], [??]. В общем виде модель должна аппроксимировать условное распределение $p(y \mid x)$, где x — текстовое условие, а y — MIDI-фрагмент.

Эта постановка отличается от безусловной генерации музыки: недостаточно получить правдоподобную музыкальную последовательность, её нужно ещё согласовать с запросом. Она также отличается от задач, где условием служит уже заданный музыкальный материал, например мелодия или аккордовая последовательность [??], [??]. В text-to-midi условие принадлежит языковой модальности, поэтому модель должна связать слова о жанре, настроении, темпе, размере, инструментах и фактуре с конкретными музыкальными событиями.

Для задачи принципиальна неоднозначность. Один текстовый запрос может иметь несколько корректных музыкальных реализаций, поэтому качество нельзя свести к точному совпадению с единственным эталоном. Именно поэтому далее, в разделе ??, модель дообучается не по готовой целевой MIDI-последовательности, а по относительному качеству нескольких сгенерированных кандидатов.

4 Обзор существующих решений

Развитие text-to-midi связано с более общим переходом от безусловной символической генерации к управляемой генерации музыки. Сначала управление задавалось структурированными признаками: эмоцией, стилем, темпом, аккордовой основой или другими атрибутами [??], [??]. Такие признаки проще извлекать из данных, но они хуже соответствуют пользовательскому сценарию, где удобнее задавать музыку свободным текстом.

Ранние работы уже связывали текст и символическую музыку, но чаще в более узкой постановке. *TeleMelody* использует текст как источник шаблонов для генерации мелодии, а *BUTTER* рассматривает совместное представление музыкальных фрагментов и предложений [??], [??]. Эти работы важны как переход от жёстких атрибутов к языковому условию, но они ещё не решают задачу прямой многорожечной MIDI-генерации по произвольному описанию.

Дальнейшие подходы различаются тем, как именно связываются текст и музыка. В *MuseCoco* текст сначала переводится в промежуточные музыкальные атрибуты, после чего

отдельная модель генерирует музыку по этим атрибутам [??]. В работах, использующих ABC-нотацию [??], музыка представляется как текстово-совместимая строка, что упрощает применение языковых моделей [??], [??]. Однако ABC не совпадает с прямой многодорожечной MIDI-генерацией. Для прикладного редактирования и анализа инструментальных партий MIDI остаётся более естественным форматом.

Современная задача прямого преобразования текста в MIDI опирается на специальные корпуса и токенизированные музыкальные представления. Крупные MIDI-корпуса, такие как *Lakh MIDI Dataset*, дают исходный музыкальный материал, но сами по себе почти не содержат языковых описаний [??]. Эту проблему закрывают специализированные ресурсы: *MidiCaps* содержит текстовые описания MIDI-файлов, а *Text2midi* опирается на предварительное обучение на символьной музыке и последующее дообучение на парах “текст–MIDI” [??], [??]. Близкие системы, такие как *ChatMusician* и *MeloTrans*, показывают, что языковые модели и архитектуры преобразователей могут работать с символьной музыкой, но при этом выбор представления и способ обучения существенно влияют на управляемость результата [??], [??].

Главные ограничения существующих решений связаны с данными и структурой музыки. Парные наборы “текст–MIDI” меньше и менее однородны, чем корпуса обычного текста или аудио. Кроме того, хорошее предсказание локальных токенов не гарантирует согласованности целого фрагмента: модель может создавать лишние дорожки, чрезмерно плотную фактуру, слабую гармоническую поддержку или материал, слабо соответствующий высокоуровневому смыслу запроса. Эти ограничения подводят к задаче последующего согласования модели с более сложными критериями качества.

5 Мотивация последующего дообучения

При обычном контролируемом дообучении модель учится предсказывать следующий токен по уже сгенерированному контексту. Для text-to-midi это важно: такая цель помогает модели сохранять синтаксис MIDI-представления и не разрушать формат последовательности. Однако качество музыки не сводится к локальной вероятности отдельных токенов. Готовый MIDI-фрагмент оценивается целиком: насколько он тонально согласован, насколько устойчив его ритм, не распадаются ли партии мелодии и сопровождения, соответствует ли результат текстовому описанию.

После базового обучения остаётся задача согласовать модель с музыкальными критериями, которые проявляются только на уровне целого фрагмента. Похожая логика встречается и в других работах: *RL-Duet* рассматривает генерацию аккомпанемента как задачу обучения с подкреплением [??], *MusicRL* использует человеческие предпочтения для text-to-audio генерации [??], а *NotaGen* и *SMART* показывают применимость внешних критериев к символьной музыке [??], [??]. Для text-to-midi близкий подход предложен в *Text2midi-InferAlign*, где награда включается уже на этапе поиска при генерации [??].

В этой работе согласование выполняется поверх уже обученной text-to-midi модели. Для одного запроса генерируется несколько вариантов, каждый вариант оценивается как завершённый MIDI-фрагмент, после чего модель обновляется в сторону относительно более удачных генераций. Для такой постановки подходит GRPO: метод сравнивает ответы внутри одной группы и не требует обучения отдельной модели ценности.

6 Предлагаемый подход

6.1 Постановка задачи и общая идея

Исходная text-to-midi модель не обучается заново. Дообучение применяется как надстройка над уже обученным генератором и использует дополнительные критерии качества. Для MIDI это существенно: локальная корректность токенов ещё не означает, что весь фрагмент тонально, ритмически и фактурно согласован. Награда поэтому вычисляется после декодирования полной музыкальной последовательности.

Для одного текстового запроса модель генерирует несколько MIDI-кандидатов. Затем кандидаты оцениваются метриками на основе правил или внешним критиком, а параметры модели обновляются по относительному качеству вариантов внутри группы. Такая постановка близка к согласованию с предпочтениями в музыкальной генерации [??], [??] и к последующему согласованию

text-to-midi моделей [??].

6.2 Базовая модель Text2midi

Исходным генератором служит модель семейства *Text2midi*, предназначенная для прямого преобразования текстового описания в символьную музыкальную последовательность [??]. На вход модель получает текстовый запрос на естественном языке. В запросе могут быть указаны такие свойства, как жанр, настроение, темп, размер, тональность, инструментальный состав и общая фактура. На выходе модель авторегрессионно порождает последовательность музыкальных токенов, которая затем декодируется в MIDI-файл.

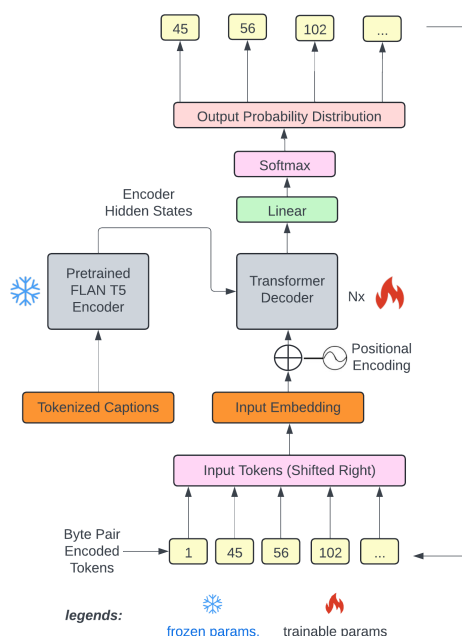


Рисунок 6 – Схема архитектуры Text2midi [??].

В дальнейшем модель трактуется как условная политика $\pi_{\theta}(y \mid x)$, где x — текстовый запрос, а y — сгенерированная MIDI-последовательность. Базовая модель уже умеет порождать валидные MIDI, но может создавать лишние дорожки, слишком плотные фрагменты, большое число ударных событий или материал, слабо согласованный с текстовым запросом. Дообучение должно сместить распределение ответов в сторону более предпочтительных кандидатов, не разрушая исходную способность генерировать символьную музыку.

6.3 GRPO как схема последующего дообучения

Для последующего дообучения выбран *Group Relative Policy Optimization* (GRPO) [??]. В text-to-MIDI эта схема удобна по двум причинам: качество фрагмента оценивается после завершения генерации, а один запрос допускает несколько музыкально корректных ответов. GRPO не требует единственного эталона; вместо этого он сравнивает ответы модели между собой и усиливает варианты с более высокой оценкой внутри группы.

Пусть q обозначает текстовый запрос, а o_i — i -ю сгенерированную последовательность MIDI. На каждом шаге текущая политика, зафиксированная на момент генерации, порождает группу из G ответов:

$$\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O \mid q).$$

Каждый ответ декодируется в MIDI и получает скалярную награду $r_i = R(q, o_i)$. Награда вычисляется для всей последовательности: её источником могут быть музыкальные правила, оценка внешнего критика или их комбинация. Относительное преимущество ответа определяется нормировкой наград внутри группы:

$$\hat{A}_i = \frac{r_i - \bar{r}}{s_r + \varepsilon_A}, \quad \bar{r} = \frac{1}{G} \sum_{j=1}^G r_j, \quad s_r = \text{std}(r_1, \dots, r_G).$$

Так как награда относится к завершённому MIDI-фрагменту, одно и то же значение $\hat{A}_i$ применяется ко всем токенам последовательности o_i . Если в одном шаге используется несколько запросов, нормировка выполняется отдельно для группы каждого запроса.

Для токена $o_{i,t}$ вводится отношение вероятностей новой и старой политики:

$$\rho_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}.$$

Отношение показывает, насколько обновляемая модель изменила вероятность токена относительно политики, использованной при генерации текущей группы. Для ограничения размера обновления применяется отсечение по интервалу $[1 - \epsilon, 1 + \epsilon]$. В реализации минимизируется следующая функция потерь:

$$\mathcal{L}_{\text{GRPO}} = \frac{1}{\sum_{i,t} m_{i,t}} \sum_{i,t} m_{i,t} \left(-\min(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i) + \beta D_{i,t} \right),$$

где $m_{i,t}$ — маска валидных токенов, ϵ задаёт допустимое изменение политики за один шаг, а β определяет вес регуляризации. Слагаемое $D_{i,t}$ удерживает модель рядом с фиксированной опорной

моделью π_{ref} :

$$D_{i,t} = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})} - 1.$$

На практике эта величина вычисляется через логарифмы вероятностей токенов. Если ответ имеет положительное относительное преимущество, обновление увеличивает вероятность составляющих его токенов; если преимущество отрицательное, вероятность таких токенов уменьшается. KL-регуляризация удерживает модель от слишком далёкого отклонения от исходного генератора.

Особенности применения GRPO к text-to-MIDI приведены в таблице ??.

Таблица 14 – Особенности применения GRPO к задаче text-to-MIDI

Элемент	Реализация в работе
Единица сравнения	Завершённая MIDI-последовательность, полученная после декодирования токенов.
Группа ответов	Несколько вариантов для одного текстового запроса; преимущества считаются внутри этой группы.
Награда	Скалярная оценка MIDI-фрагмента, вычисляемая правилами, критиком или их комбинацией.
Сигнал критика	В экспериментах используется сырая оценка критика, ранг внутри группы или доля попарных побед.
Ограничение обновления	Отсечение отношения вероятностей снижает риск резкого изменения политики.
Опорная модель	KL-слагаемое ограничивает отклонение от исходного генератора.
Валидность	Невалидные или вырожденные MIDI-фрагменты получают штраф и не должны усиливаться при обучении.

6.4 Один шаг обучения в реализованной процедуре

Процедура обучения строится вокруг одного итерационного шага GRPO. На вход шага поступает один или несколько текстовых запросов. В зависимости от конфигурации они могут выбираться из датасета, генерироваться синтетически по шаблону или фиксироваться заранее для диагностического эксперимента. Для каждого запроса модель генерирует несколько независимых кандидатов с одинаковым текстовым условием. Один шаг обучения поэтому содержит не отдельный ответ модели, а группу вариантов для последующего сравнения.

Стадии одного шага сведены в таблицу ??. Ключевая особенность процедуры состоит в отложенной награде: качество оценивается не во время выбора отдельного токена, а после декодирования всей последовательности.

В реализации дополнительно сохраняются служебные артефакты каждого шага: тексты запросов, MIDI-файлы сгенерированных кандидатов, значения компонент награды, статистика валидности, контрольные точки модели и метрики обучения. Такие артефакты нужны не только для

Таблица 15 – Один шаг GRPO-дообучения в реализованной процедуре

Этап	Операция	Назначение
1	Выбор запроса	Задаёт текстовое условие, для которого будет построена группа кандидатов.
2	Генерация кандидатов	Модель порождает несколько MIDI-кандидатов при одном и том же условии.
3	Декодирование	Токены преобразуются в музыкальное представление и MIDI-файл.
4	Вычисление награды	Кандидат оценивается компонентами на основе правил или внешним критиком.
5	Нормировка внутри группы	Награды превращаются в относительные преимущества кандидатов.
6	Обновление GRPO	Обновляются обучаемые параметры с отсечением отношения вероятностей и KL-регуляризацией.

воспроизводимости, но и для анализа поведения модели. В задачах музыкальной генерации численная награда не всегда полностью объясняет воспринимаемое качество, поэтому возможность прослушать сохранённые MIDI и сопоставить их с компонентами награды входит в экспериментальную процедуру.

6.5 Настройка обучаемых параметров модели

Полное обновление всех весов в режиме обучения с подкреплением дорого и рискованно: награда шумна, а сильное изменение параметров может ухудшить валидность MIDI. По этой причине используются ограниченные режимы настройки параметров (таблица ??).

Таблица 16 – Режимы настройки обучаемых параметров

Режим	Что обучается	Роль в экспериментах
LoRA-адаптация [??]	Низкоранговые добавки к выбранным линейным слоям	Основной экономичный режим для GRPO-дообучения.
Прямое частичное дообучение	Непосредственно выбранные параметры исходной модели	Более сильный, но потенциально менее устойчивый вариант.

На практике основным стал вариант LoRA только в FFN-блоках: низкоранговые добавки подключаются к слоям декодера `linear1` и `linear2`. Этот режим меняет распределение музыкальных токенов, но оставляет большую часть модели замороженной. Прямое частичное дообучение рассматривается как более сильный вариант, требующий меньшего значения скорости обучения и более строгого контроля KL.

6.6 Формирование текстовых запросов

Текстовый запрос определяет группу кандидатов, внутри которой нормируются награды. Он влияет не только на генерацию, но и на разброс обучающего сигнала. В экспериментах используются короткие шаблонные запросы с явно заданными атрибутами:

A moderate warm folk piece in C major at 96 BPM,
with a 4/4 time signature. Use piano for the lead melody
and acoustic guitar for steady accompaniment.

6.7 Стратегия выбора запросов для обучения

В экспериментах предусмотрены несколько режимов подачи текстовых запросов (таблица ??). Фиксированный запрос нужен для диагностики обучаемости сигнала награды, а более широкие режимы — для проверки обобщения.

Таблица 17 – Режимы формирования текстовых запросов в GRPO-экспериментах

Режим	Что фиксируется	Зачем используется
Фиксированный запрос	Все текстовые атрибуты постоянны	Диагностика обучаемости сигнала награды.
Случайные запросы	Атрибуты случайно выбираются из шаблона	Проверка обобщения на разные условия.
Учебная последовательность	Пространство запросов расширяется постепенно	Снижение шума и плавное усложнение задачи.
Смешанный режим	Часть запросов фиксирована, часть случайна	Совмещение диагностики и обобщения.

Дополнительно вводится учебная последовательность: сначала фиксируется простой запрос, затем поэтапно меняются тональность, размер и стиль. Такой порядок уменьшает разброс награды между шагами и помогает проверить, не забывает ли модель уже освоенное поведение.

6.8 Проектирование функции награды

Функция награды подбиралась итеративно. Сначала проверялись интерпретируемые MIDI-правила, затем критик v1 и критик v2, после этого смешанные схемы. Подход согласуется с работами, где обучение с подкреплением применяется к музыкальной генерации через внешние эстетические или предпочтительные оценки [??], [??], [??]. Семейства сигналов награды приведены в таблице ??.

Таблица 18 – Семейства функций награды

Семейство	Источник оценки	Роль в работе
Символьные правила	Метрики, вычисляемые по MIDI-структуре	Контроль интерпретируемых музыкальных свойств.
Сигналы критика	Критик v1/v2	Приближение более сложного музыкального предпочтения.
Смешанные схемы	Взвешенная комбинация правил и критика	Стабилизация оценки критика через простые ограничения.

6.9 Структура используемых компонент награды

6.9.1 Общие обозначения и нормировка

Все компоненты на основе правил вычисляются после декодирования токенов в символьную MIDI-структуру. Обозначим множество нот через N , число нот через $|N|$, длительность фрагмента в долях через D , число непустых дорожек через C , долю барабанных нот через ρ_d , а текстовый запрос через x . Для мягких ограничений используется треугольная нормировка:

$$T(z; l, c, h) = \begin{cases} 0, & z \leq l \text{ или } z \geq h, \\ \frac{z - l}{c - l}, & l < z < c, \\ 1, & z = c, \\ \frac{h - z}{h - c}, & c < z < h. \end{cases}$$

Эта функция задаёт целевой диапазон значений. Максимальная награда достигается в точке c ; отклонение в сторону слишком малых или слишком больших значений линейно уменьшает вклад соответствующей компоненты.

6.9.2 Тональная согласованность

Компонента тональности использует длительно-взвешенный профиль высот p и сравнивает его с профилем тональности, заданной текстовым запросом. Если k_x — целевой профиль, то

$$a = \frac{\text{corr}(p, k_x) + 1}{2}, \quad b = \frac{\text{corr}(p, k_x) - \max_{k \neq k_x} \text{corr}(p, k) + 1}{2},$$

$$R_{\text{key}} = \text{clip}(0.75a + 0.25b, 0, 1).$$

Первое слагаемое отражает абсолютное совпадение с целевой тональностью, второе — отделимость целевой тональности от альтернативных тональностей.

6.9.3 Метрическая организация

Компонента метра оценивает не только записанный размер, но и расположение акцентов в нотном материале. Для каждого кандидата вычисляется соответствие f_m нескольким метрическим шаблонам, после чего целевой размер из текстового запроса получает оценку

$$R_{\text{meter}} = \text{clip}\left(0.7 \frac{f_x}{\max_m f_m} + 0.3 f_x, 0, 1\right).$$

Эта формула учитывает абсолютную выраженность целевого метра и его относительное преимущество перед другими возможными размерами.

6.9.4 Длительность и плотность нот

Компоненты длительности и плотности также задаются через треугольную нормировку:

$$R_{\text{dur}} = T(D; 8, 64, 160), \quad R_{\text{dens}} = T\left(\frac{|N|}{\max(D, 1)}; l, c, h\right).$$

Компонента R_{dur} снижает награду для слишком коротких фрагментов и чрезмерно длинных артефактов. Компонента R_{dens} контролирует число нот на одну долю. Параметры l, c, h задаются в конфигурации эксперимента; так можно отдельно исследовать обычную и умеренную плотность.

6.9.5 Дорожки и ударные инструменты

Аранжировочные ограничения задаются мягкими функциями. Компонента числа дорожек поощряет непустые MIDI с числом дорожек не больше C_{max} , а при превышении лимита линейно убывает. Для ударных используется доля барабанных нот:

$$R_{\text{drum}} = \begin{cases} 1, & \rho_d \leq a, \\ 0, & \rho_d \geq b, \\ 1 - \frac{\rho_d - a}{b - a}, & a < \rho_d < b. \end{cases}$$

Такое правило не запрещает ударные полностью, но снижает награду, если они начинают доминировать в MIDI-фрагменте.

6.9.6 Валидность и внешний критик

Проверка валидности служит защитным механизмом. Если MIDI не декодируется, содержит слишком мало нот, слишком короткий фрагмент или чрезмерное повторение одной высоты, выставляется индикатор $I_{\text{bad}} = 1$. В экспериментах с внешним критиком дополнительно вводится сигнал R_{critic} : сырая оценка, ранг внутри группы или доля попарных побед кандидата над другими вариантами.

6.9.7 Итоговая функция награды

Итоговая награда задаётся как взвешенная сумма активных компонент:

$$R(x, y) = \begin{cases} -p_{\text{bad}}, & I_{\text{bad}}(y) = 1 \text{ и включён жёсткий фильтр,} \\ \sum_k \lambda_k R_k(x, y) + \lambda_c R_{\text{critic}}(x, y), & \text{иначе.} \end{cases}$$

Здесь p_{bad} — штраф за невалидный MIDI, λ_k — веса компонент на основе правил, а λ_c — вес внешнего критика. В режиме обучения по критiku ранговый сигнал особенно полезен тогда, когда абсолютные значения критика плохо калиброваны, но порядок кандидатов достаточно информативен.

6.10 Конфликты между компонентами награды

Добавление новых правил не всегда улучшает обучение: компоненты награды могут конфликтовать. Это проверялось в пробных экспериментах с разными наборами компонент: тональностью и метром, плотностью нот, ограничениями на дорожки, запретом ударных и сигналами критика. После этих запусков часть бинарных правил была заменена мягкими метриками, а веса наиболее жёстких компонент были снижены. Выявленные конфликты приведены в таблице ??.

Состав функции награды уточнялся эмпирически: анализировались не только суммарный сигнал и вклад отдельных компонент, но и сохранённые MIDI-примеры.

Таблица 19 – Типичные конфликты между компонентами награды

Конфликт	Причина	Практическое решение
Плотность и естественность	Высокая плотность улучшает формальную заполненность, но может звучать перегруженно.	Переход к умеренной плотности.
Метр и свобода фразы	Шаблон метра помогает контролю, но может усиливать механичность.	Ослабление веса или временное отключение компоненты.
Запрет барабанов и обучаемость	Бинарный сигнал часто одинаков для всех кандидатов.	Замена на долю барабанных нот.
Число дорожек и полнота аранжировки	Жёсткое ограничение упрощает MIDI, но может удалить содержательные партии.	Мягкие ограничения и анализ сохранённых MIDI.

6.11 Ожидаемые преимущества и исследовательские гипотезы

Гипотезы работы сведены в таблицу ??.

Таблица 20 – Ожидаемые эффекты предлагаемого подхода

Гипотеза	Ожидаемый эффект
GRPO применим к text-to-midi	Модель сможет усиливать более удачные MIDI-кандидаты без эталонной последовательности.
Ограниченная настройка параметров достаточна	LoRA или частичное дообучение смогут изменить поведение модели без полного обновления всех весов.
Мягкие компоненты награды лучше бинарных	Непрерывные сигналы будут чаще различать кандидатов внутри группы.
Узкое распределение запросов снижает шум	Фиксированный запрос и учебная последовательность позволят яснее увидеть динамику обучения.
Внешний критик может служить источником награды	Сигнал критика позволит согласовывать модель с более сложными критериями качества.

7 Экспериментальная часть

7.1 Цели экспериментального исследования

Экспериментальная часть проверяет, можно ли использовать предложенную схему для последующего дообучения text-to-MIDI модели. В отличие от стандартного обучения с эталонной MIDI-последовательностью, здесь модель получает только текстовый запрос и несколько собственных вариантов ответа. Главный вопрос состоит в том, достаточно ли относительных оценок MIDI-кандидатов для устойчивого обновления модели.

Первая цель — проверить саму возможность дообучения с подкреплением для символьной музыкальной генерации. Необходимо установить, приводит ли обновление параметров по групповым оценкам к изменению поведения модели, не разрушая при этом способность генерировать валидные MIDI-файлы. Для этого анализируются значения итоговой награды, доля валидных генераций, характер ошибок декодирования, динамика KL-штрафа и изменение отдельных компонент награды.

Вторая цель — сравнить источники награды: интерпретируемые правила, критик v1/v2 и смешанные схемы. Компоненты на основе правил контролируют простые музыкальные свойства: тональность, метр, длительность, плотность нот и долю ударных событий. Критик, напротив, должен приближать более сложное предпочтение, которое трудно выразить набором простых формул. В этих экспериментах проверяется, какие сигналы дают различимые оценки внутри группы кандидатов и какие из них приводят к более устойчивому обновлению модели.

Третья цель — выбрать режим адаптации параметров. Полное дообучение модели в режиме обучения с подкреплением вычислительно дорого и может приводить к нестабильным изменениям политики. Сравнение ведётся между более ограниченными режимами: LoRA-адаптацией выбранных слоёв, настройкой FFN-блоков и частичным дообучением. Здесь важен не только рост награды, но и устойчивость обучения: отсутствие резких скачков функции потерь, чрезмерного роста KL и деградации качества MIDI.

Ещё одна задача — сопоставить внутренние метрики обучения с внешней оценкой качества. Рост суммарной награды сам по себе не гарантирует улучшения музыкального результата: модель может переоптимизироваться под отдельную компоненту, например чрезмерно увеличить плотность нот или механически следовать метрическому шаблону. Итоговая оценка строится не только по логам обучения, но и по построенному A/B-сравнению базовой и дообученной модели, а также по оценкам внешней модели-оценщика.

7.2 Инфраструктура и программная реализация

Экспериментальная инфраструктура реализована как расширение открытого проекта Text2MIDI¹. В кодовую базу добавлены процедура последующего GRPO-дообучения, модуль вычисления наград, система конфигураций и средства сохранения результатов. Реализация организована модульно: отдельно задаются конфигурации запусков, процедура обучения, компоненты функции награды и инструменты последующего анализа.

Программная часть написана на языке Python [??], а обучение нейросетевых моделей и вычисление градиентов выполняются с использованием PyTorch [??]. Этот стек даёт возможность гибко управлять генерацией последовательностей, повторно вычислять логарифмы вероятностей токенов для GRPO и сохранять состояние оптимизатора между запусками.

Обучение запускается по имени эксперимента. Для каждого запуска создаётся YAML-конфигурация, в которой фиксируются параметры GRPO: число текстовых запросов на шаг, число генерируемых кандидатов, максимальная длина генерации, скорость обучения, веса компонент награды и режим адаптации параметров модели. Часть параметров можно переопределить из командной строки; это упрощает проведение серий сопоставимых экспериментов.

Во время обучения сохраняются контрольные точки модели, значения отдельных компонент награды, служебные метрики оптимизации и, при необходимости, промежуточные MIDI-кандидаты. Логирование ведётся в Weights & Biases [??]: по этим логам сопоставляются динамика суммарной награды с поведением отдельных компонент, KL-штрафом, функцией потерь, нормами градиентов и статистикой валидности. Логи запусков и связанные с ними артефакты экспериментов доступны в материалах репозитория.

7.3 Семейства исследуемых конфигураций

Эксперименты организованы как несколько семейств конфигураций, а не как один длинный запуск. Такое разделение помогает отдельно оценить влияние источника награды, длины генерации, способа выбора текстовых запросов и набора обучаемых параметров. Базовая модель во всех случаях остаётся одной и той же, а различия между запусками задаются через конфигурационные файлы.

Первое семейство составляют запуски с наградами на основе явно заданных музыкальных правил. В них модель получает сигнал за соответствие тональности, метрической структуре, умеренной плотности нот, балансу длительности и ограничению доли ударных событий. Эти эксперименты показывают, может ли GRPO улучшать формальные свойства MIDI без привлечения

¹blank

обученного музыкального критика.

Во втором семействе основной наградой служит только критик выбранной версии. В таких запусках качество не задаётся вручную в виде отдельных компонент; модель получает скалярный сигнал, полученный из ранжирования нескольких сгенерированных вариантов. Здесь проверяется, достаточно ли обобщённого сигнала качества для устойчивого дообучения генератора.

Третье семейство объединяет оба подхода: часть награды задаётся правилами, а часть — критиком. Такие конфигурации рассматриваются как компромисс между интерпретируемостью ручных признаков и более общей оценкой музыкальности. Предварительные запуски показали, что слишком большое число одновременно активных компонент может давать конфликтующие градиенты; поэтому смешанные конфигурации проверялись в нескольких вариантах с разными весами.

Также рассматривались эксперименты с отбором дорожек после генерации. В них из исходного MIDI выбирались наиболее информативные неударные дорожки, после чего уже этот вариант передавался в вычисление награды. Эти запуски отделяют качество музыкального материала от шума, возникающего из-за пустых, слишком коротких или преимущественно ударных дорожек. Позднее проверялись и конфигурации без такого отбора, чтобы оценить, способна ли модель научиться генерировать более подходящую структуру напрямую.

Ещё одна группа экспериментов связана с ограничением пространства текстовых запросов. Вместо немедленного обучения на широком наборе описаний использовались фиксированный запрос и небольшой учебный набор, в котором постепенно менялись тональность, размер и стиль. Учебная последовательность нужна для диагностики: если модель не демонстрирует изменений даже на узком классе запросов, то расширение набора описаний не устраняет проблему, а лишь увеличивает шум оценки.

7.4 Конфигурации текстовых запросов и режимов генерации

В GRPO один шаг обучения строится не по отдельной сгенерированной последовательности, а по группе кандидатов для одного или нескольких текстовых описаний. Если на шаге используется P описаний и для каждого описания генерируется G вариантов, то всего на шаг получается $P \cdot G$ MIDI-кандидатов. Награды нормируются внутри группы вариантов, относящихся к одному и тому же описанию, поэтому выбор P и G напрямую влияет на шумность обучающего сигнала.

В начальных экспериментах использовались более широкие наборы описаний: несколько запросов на шаг и небольшое число кандидатов для каждого из них. Этот режим быстрее покрыв-

Таблица 21 – Основные семейства экспериментальных конфигураций

Семейство	Основная идея	Что проверяется
Правила	Награда из музыкальных признаков: тональность, метр, плотность, длительность, доля ударных	Способность GRPO улучшать явно заданные свойства MIDI
Критик v1/v2	Награда строится по ранжированию сгенерированных вариантов	Достаточность обученного сигнала качества без ручных правил
Правила + критик v1/v2	Сумма интерпретируемых компонент и оценки критика	Баланс между устойчивостью правил и обобщённой музыкальной оценкой
Отбор дорожек	Награда считается после выбора информативных неударных дорожек	Влияние пустых и шумовых дорожек на обучение
Учебная последовательность запросов	Постепенное усложнение: базовое описание, тональность, размер, стиль	Возможность увидеть эффект обучения на узком и контролируемом наборе условий

вает разные музыкальные условия, но увеличивает разброс наград: модель одновременно видит разные тональности, размеры, стили и требования к фактуре. Для ранней проверки работоспособности алгоритма это полезно, однако по таким кривым сложнее понять, действительно ли модель подстраивается под награду или изменения объясняются случайностью генерации.

В последующих диагностических запусках пространство запросов было сужено. Главный диагностический режим — один запрос на шаг и увеличенное число кандидатов. Например, в конфигурациях с 18 вариантами на один запрос один шаг обучения содержит 18 MIDI-кандидатов, а в запусках с критиком — 25 кандидатов. Это делает относительное сравнение внутри группы более устойчивым: все варианты отвечают одному и тому же текстовому условию, а различия в награде сильнее связаны с качеством самой генерации.

Максимальная длина генерации задаёт верхнюю границу числа генерируемых токенов. Для быстрых диагностических экспериментов использовалась длина 800: она даёт достаточно содержательный музыкальный фрагмент, но не делает генерацию чрезмерно дорогой. Проверялись и более длинные варианты, однако увеличение длины заметно повышает время одного шага и может усиливать шум награды, если модель начинает генерировать слишком плотный или неустойчивый материал.

Параметр `generation_chunk_size` отвечает только за техническое разбиение генерации на части. Он не меняет число кандидатов в GRPO-группе, но помогает подстроить запуск под доступную видеопамять. Параметр `update_mini_batch` аналогично задаёт размер части кандидатов, используемой при вычислении обновления. В основных диагностических запусках оба параметра

выбирались равными числу кандидатов, чтобы весь набор вариантов для одного запроса обрабатывался без дополнительного дробления.

Для проверки способности модели удерживать простые изменения условий была добавлена учебная последовательность запросов. Она состоит из четырёх этапов: сначала фиксируется базовое описание, затем меняется тональность, после этого размер, а на последнем этапе — стиль. При переходе к новым этапам часть ранних запросов повторяется, чтобы уменьшить забывание уже освоенных условий. Этот режим не предназначен для финальной оценки качества на широком наборе описаний; его задача — показать, появляется ли управляемое изменение поведения модели в простой и контролируемой постановке.

7.5 Сравнимые стратегии адаптации параметров

Исходная Text2MIDI-модель уже обучена генерировать MIDI, поэтому полное переобучение всех параметров не проводится. Вместо этого сравниваются ограниченные режимы настройки, в которых изменяется только небольшая часть модели. Такой выбор снижает вычислительные требования и уменьшает риск разрушить уже выученную способность генерировать синтаксически корректные MIDI-последовательности.

Главный режим основан на LoRA [??]: веса исходной модели замораживаются, а к выбранным линейным преобразованиям добавляются обучаемые низкоранговые матрицы. В экспериментах проверялись три варианта такого подключения. Первый вариант изменяет только слои внимания (`to_qkv`, `to_out`); он потенциально влияет на связи между токенами и структуру последовательности. Второй вариант изменяет только FFN-блоки декодера (`linear1`, `linear2`); он оказался более удобным для локального смещения распределения музыкальных токенов без резкой перестройки внимания. Третий вариант объединяет оба набора слоёв и даёт модели больше свободы, но требует более осторожного выбора скорости обучения и KL-ограничения.

Также рассматривалось прямое частичное дообучение. В этом режиме LoRA-добавки не используются: выбранные параметры самой модели становятся обучаемыми, а остальные веса остаются замороженными. Этот вариант сильнее воздействует на модель, из-за чего для него применялись меньшие значения скорости обучения и более строгий контроль KL. На практике он полезен как проверка того, не ограничивает ли LoRA слишком сильно пространство возможных изменений.

Для итоговых серий наиболее важным оказался режим LoRA на FFN-блоках. Он сохраняет базовую структуру генератора и при этом заметно меняет вероятности локальных музыкальных решений: выбор длительностей, плотность нот, распределение инструментальных событий и реакцию

Таблица 22 – Сравнимые режимы настройки параметров

Режим	Обучаемые параметры	Назначение в экспериментах
LoRA на слоях внимания	to_qkv, to_out	Проверка влияния на зависимость между токенами и организацию последовательности
LoRA на FFN	linear1, linear2	Основной устойчивый режим для поздних запусков с правилами и критиком
LoRA на слоях внимания и FFN	Все перечисленные выше слои	Более гибкая, но менее стабильная настройка распределения генерации
Прямое частичное дообучение	Выбранные параметры исходной модели	Проверка более сильного вмешательства при меньшей скорости обучения

на сигнал критика. В итоге этот режим использовался в учебных последовательностях запросов и в поздних экспериментах с фиксированным запросом.

7.6 Внутренние метрики и диагностика обучения

Во время обучения фиксируются итоговая награда и значения отдельных компонент. По ним можно понять, за счёт чего меняется поведение модели: например, растёт ли награда из-за лучшего соответствия тональности, из-за снижения доли ударных событий или из-за сигнала обученного критика. Для смешанных функций награды такой разбор особенно важен, поскольку рост одной компоненты может сопровождаться ухудшением другой.

Также отслеживаются служебные метрики оптимизации: функция потерь, величина KL-расхождения с исходной политикой, норма градиента и отношение вероятностей новой и старой политики. Эти величины используются как признаки устойчивости обучения. Например, резкий рост KL показывает, что модель слишком далеко отходит от исходного распределения, а большие скачки нормы градиента могут указывать на слишком высокую скорость обучения или слишком шумную награду.

Также учитываются показатели валидности MIDI-последовательностей и частота пропущенных обновлений, если соответствующая проверка включена в конфигурации. Они помогают отделить настоящий рост качества от ситуаций, когда модель получает высокую награду на технически неудачных или плохо декодируемых последовательностях.

Внутренние метрики не рассматриваются как окончательная оценка музыкального качества. Они показывают ход оптимизации, но не гарантируют, что результат стал лучше на слух. Вместе с ними проводится внешнее сравнение с базовой моделью.

7.7 Построение внешней оценки качества

Итоговая проверка качества проводится отдельно от процесса обучения. Награда участвует в обновлении модели и сама по себе не является независимым критерием. После завершения выбранного запуска сравниваются генерации базовой модели и дообученной версии на одном и том же наборе текстовых запросов.

Для каждого запроса генерируется несколько MIDI-вариантов от обеих моделей. Результаты агрегируются двумя способами: по среднему качеству нескольких генераций и по лучшему варианту из группы. Первый показатель отражает стабильность модели, второй — её способность хотя бы иногда выдавать сильный пример.

7.7.1 А/В-сравнение базовой и дообученной модели

В А/В-сравнении для каждого запроса определяется, какая из двух моделей получила более высокую агрегированную оценку. Затем считается число побед базовой и дообученной модели, доля побед и статистическая значимость по знаковому критерию. Эта схема не требует сравнивать абсолютные значения награды для разных запросов напрямую.

7.7.2 Оценка с помощью внешней языковой модели

Для дополнительной проверки MIDI переводится в ABC-нотацию [??]. Это представление сохраняет тональность, размер, голоса и последовательность нот, но остаётся текстовым и подходит для передачи в языковую модель. В роли оценщика использовалась модель DeepSeek [??]: ей передавались текстовый запрос и ABC-запись сгенерированного MIDI, после чего возвращались численная оценка и краткое объяснение.

Сравнение также проводится в формате А/В-теста. Для одного и того же запроса сопоставляются оценки генераций базовой и дообученной модели, затем агрегируются победы по запросам. Эта оценка не используется как награда во время обучения; она нужна для независимой проверки того, совпадает ли рост внутренних метрик с внешней музыкальной оценкой.

8 Результаты экспериментов

В этот раздел выносятся численные результаты и графики после завершения финальных запусков. Сначала приводится динамика обучения, затем сравнение моделей и внешняя оценка качества.

8.1 Динамика обучения

На первом этапе проверялось, способен ли предложенный контур GRPO-дообучения изменять поведение text-to-midi модели по завершённомu MIDI-фрагменту, а не только по токенам. Для этого рассматривались запуски с интерпретируемыми наградами, учебной последовательностью prompt-ов и последующим подключением критика. Динамика обучения показывает, что модель действительно реагирует на внешний сигнал: отдельные компоненты награды начинают изменяться в ожидаемом направлении, а лучшие кандидаты в группе становятся качественнее по заданным критериям.

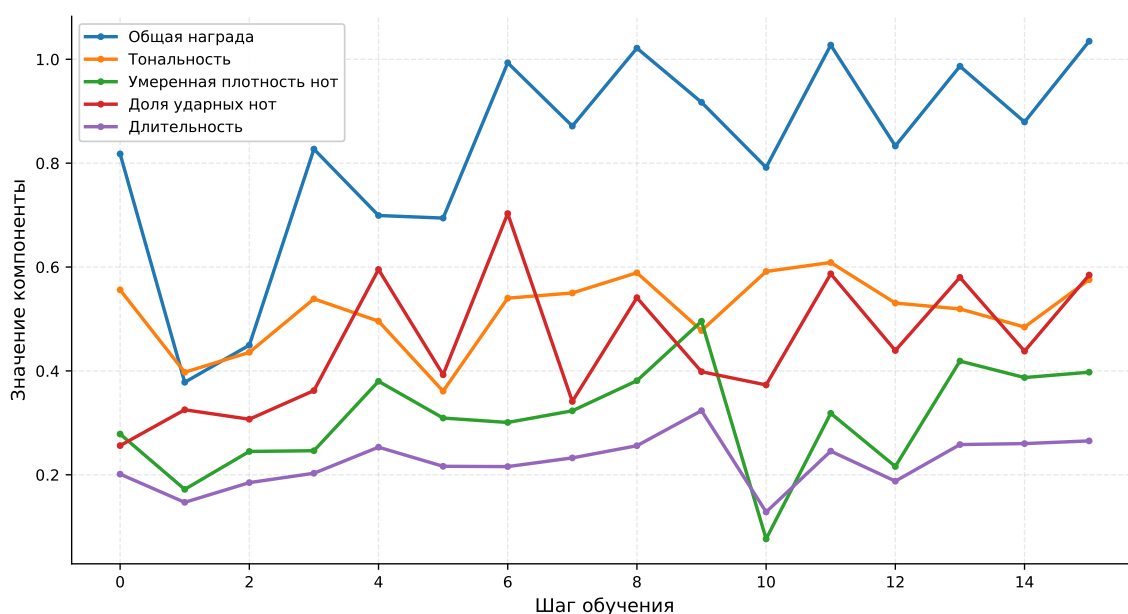


Рисунок 7 – Динамика компонент награды при обучении по правилам на узком наборе текстовых запросов.

На рис. ?? показан начальный участок обучения модели на узком наборе текстовых запросов. Уже в этом простом режиме видно, что модель подстраивается под награды, заданные на уровне MIDI: усиливается соответствие тональности, снижается доля нежелательных ударных событий, а суммарная награда растёт относительно первых итераций. Такой результат важен как техническая проверка всей схемы: награда вычисляется после декодирования MIDI, но всё равно

создаёт обучающий сигнал для языковой модели. Более шумная компонента плотности нот показывает, что не все музыкальные свойства одинаково легко оптимизируются простыми правилами.

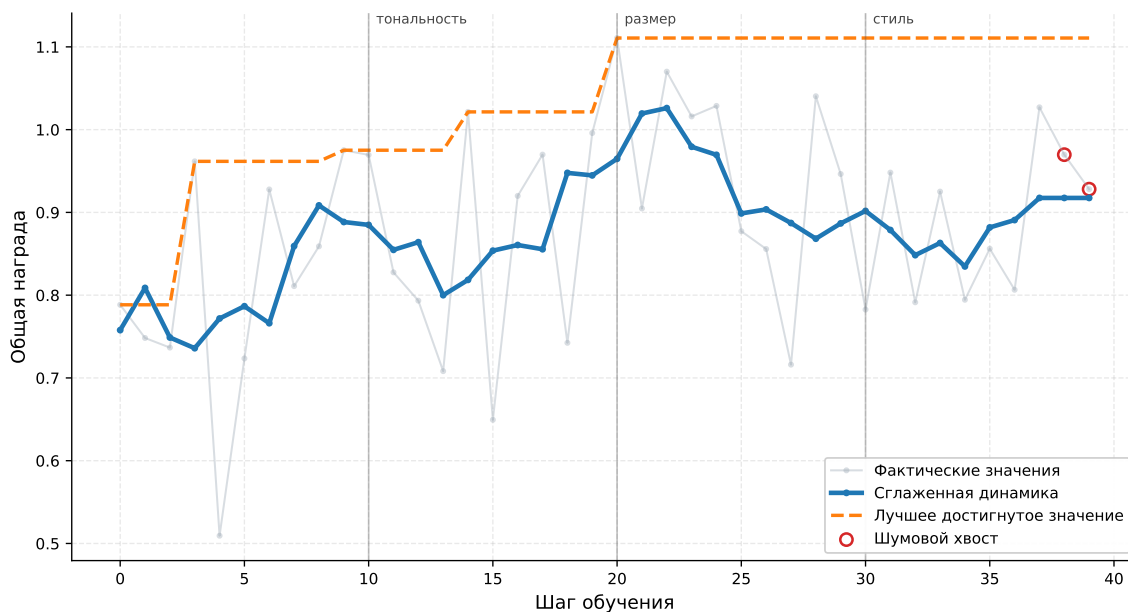


Рисунок 8 — Динамика общей награды при поэтапном обучении. Бледная линия показывает фактические значения, синяя — сглаженную динамику, оранжевая — лучшее достигнутое значение.

На рис. ?? показан запуск с учебной последовательностью запросов. Вертикальные линии отделяют этапы, на которых изменялись тональность, метрическая структура и стиль. По сравнению с немедленным обучением на разнородных prompt-ах такая схема оказалась более удобной для анализа: модель сначала закрепляет поведение на простом запросе, а затем переносит его на близкие варианты.

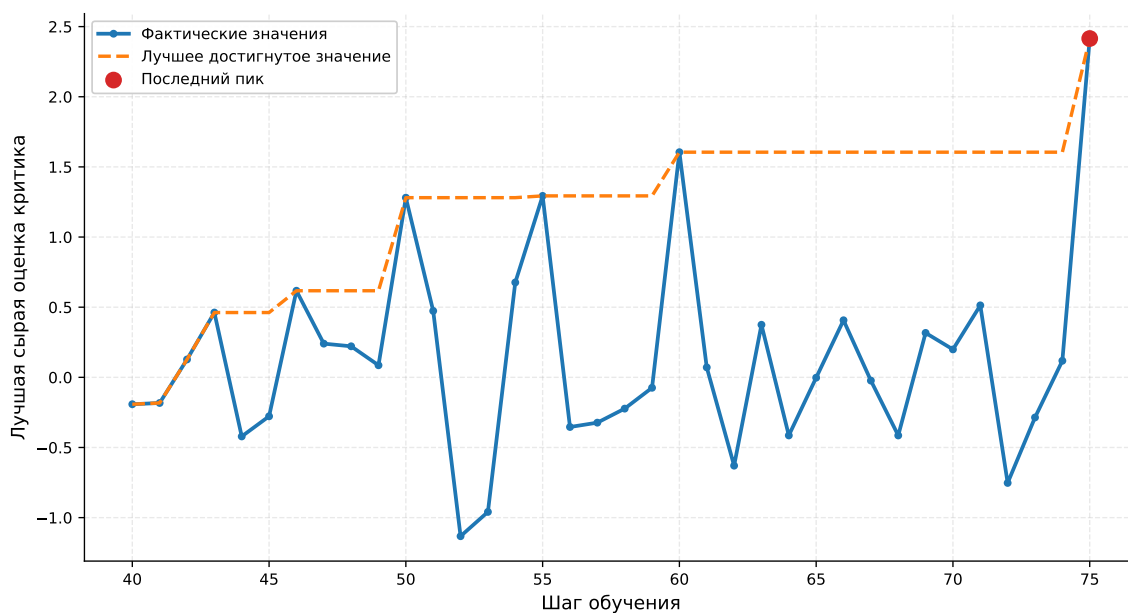


Рисунок 9 – Дообучение по критику v2 после предварительного обучения на правилах. Синяя линия соответствует фактическим значениям, оранжевая — лучшему достигнутому значению.

На рис. ?? отдельно показан этап, где модель продолжает обучение уже по сигналу критика v2. Предварительное обучение на правилах здесь играет роль стабилизации: критик v2 подключается не к полностью исходной модели, а к модели, которая уже частично соблюдает простые музыкальные ограничения. В этом режиме удаётся получить отдельные кандидаты с заметно более высокой оценкой критика v2, что подтверждает полезность последовательной схемы “правила затем критик v2”. При этом сам сигнал критика v2 остаётся более шумным, чем ручные правила; это ограничение подробнее обсуждается в разделе анализа ошибок.

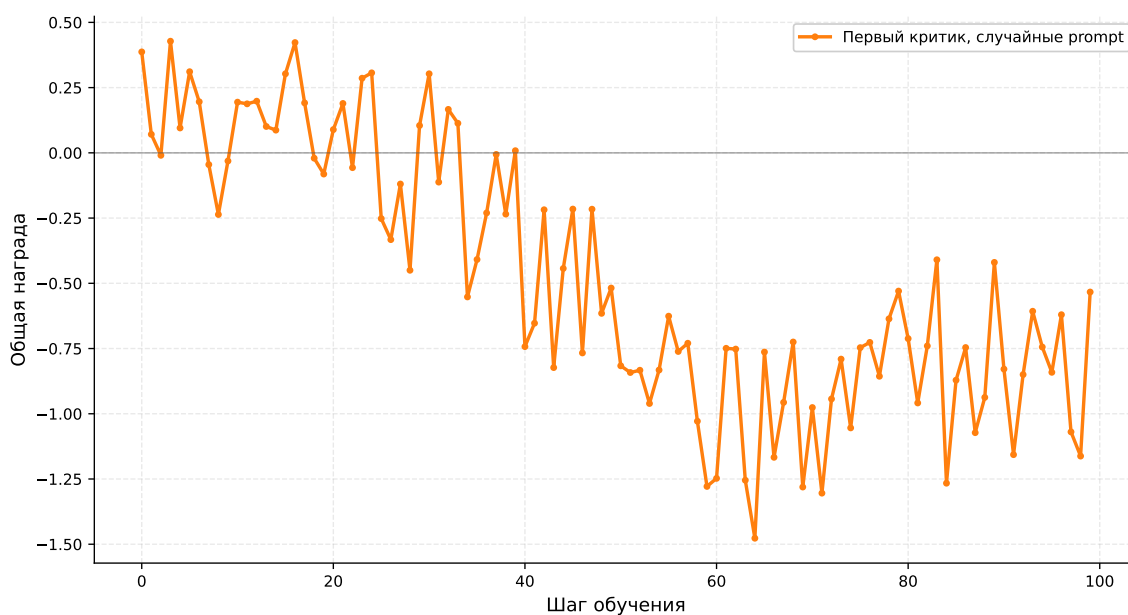


Рисунок 10 – Обучение с критиком v1 на случайных prompt-ах. Показан запуск, в котором широкое распределение запросов привело к неустойчивой динамике общей награды.

Рис. ?? приведён как контрольный пример: слишком широкое распределение запросов затрудняет обучение и делает награду менее интерпретируемой. Этот результат полезен методически, поскольку показывает, почему дальнейшие эксперименты были сосредоточены на узком наборе текстовых запросов и учебной последовательности, а не на случайном смешивании большого числа условий.

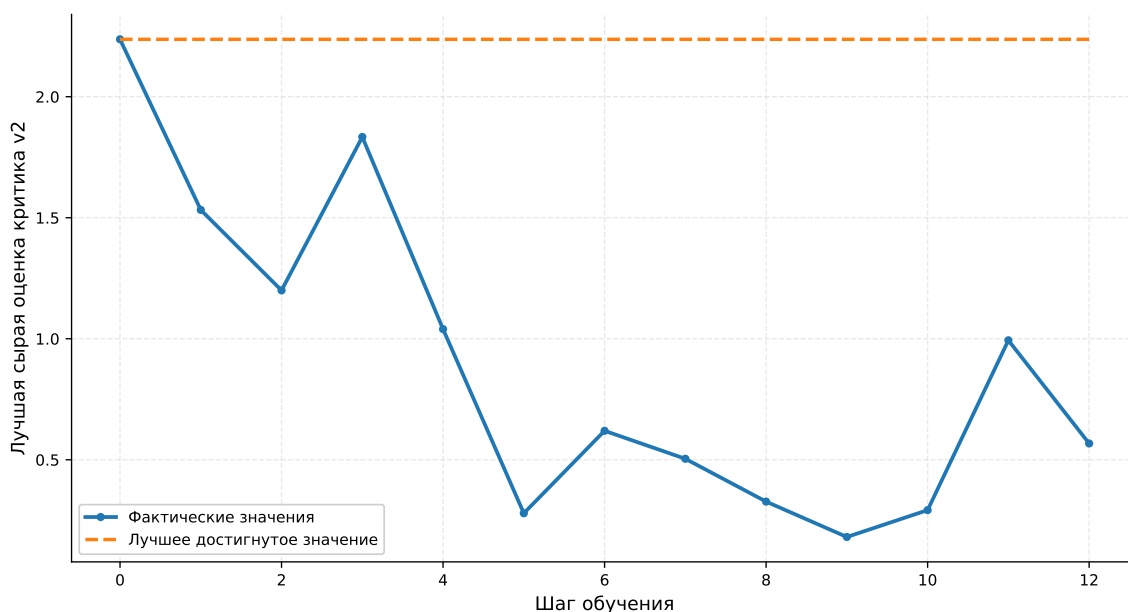


Рисунок 11 – Обучение с критиком v1 и уменьшенным шагом обучения.

На рис. ?? показан запуск дообучения модели на критике v1 на уменьшенном шаге обучения и узком наборе текстовых запросов.

8.2 А/В-сравнение

Для внутреннего сравнения использовались те же MIDI-файлы, что и для последующей внешней оценки: по 900 генераций на модель при одинаковых условиях текстового запроса. Для каждой дообученной модели использовался только тот источник оценки, по которому она обучалась: модели с правилами сравнивались по суммарной награде на основе правил, а модели с критиком — по оценке соответствующего критика. Такое сравнение не смешивает разные цели оптимизации и показывает, улучшилась ли модель именно относительно собственного обучающего сигнала.

На рис. ?? показана доля попарных побед дообученной модели над базовой; значение 0.5 соответствует паритету.

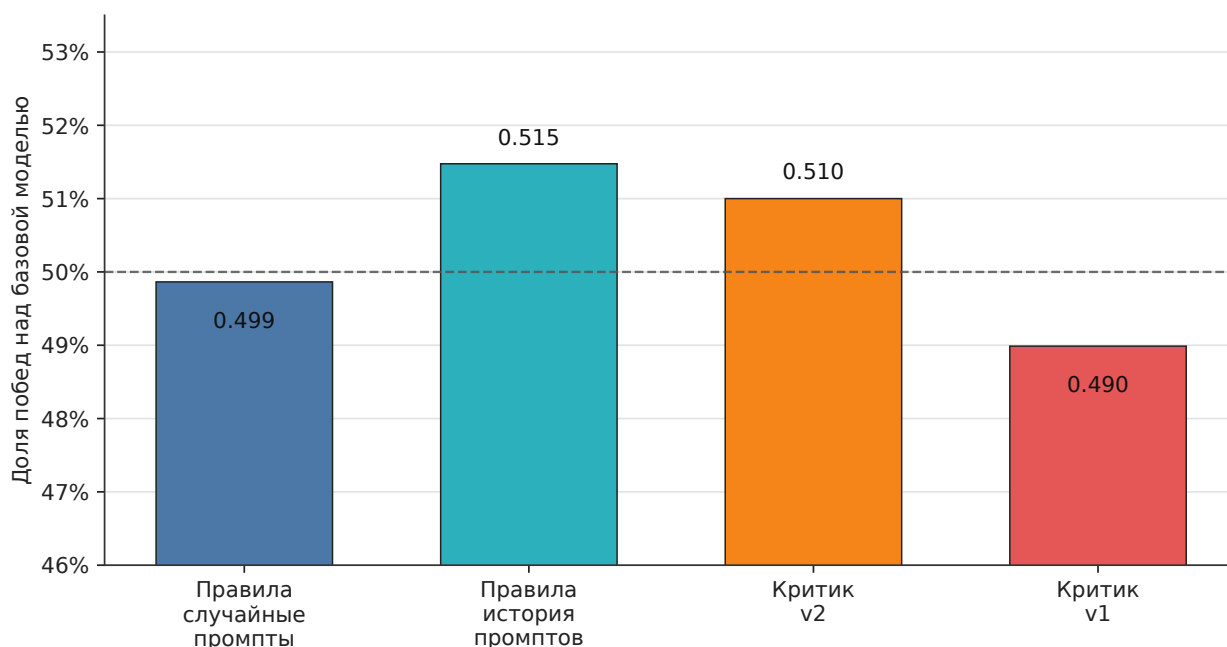


Рисунок 12 – А/В-сравнение с базовой моделью по собственному источнику награды. Показана доля попарных побед над базовой моделью.

8.3 Оценка внешней языковой моделью

Дополнительно была проведена пилотная оценка внешней языковой моделью DeepSeek v3.2 через API AITunnel. Сравнивались две модели: базовая text-to-midi модель и модель, дообученная по схеме “правила затем критик v2”. MIDI-файлы предварительно переводились в ABC, после чего в DeepSeek передавался оценочный prompt. В нём модель задавалась как эксперт по символической музыке и получала инструкцию оценить качество связи мелодии и гармонии: соответствие мелодических нот активным аккордам и тональности, поддержку сильных долей, оправданность диссонансов и хроматизмов, согласованность смены аккордов с ритмом мелодии, а также есте-

ственность басового движения. Отдельно указывалось игнорировать качество тембров, сведения, реалистичность инструментов и другие артефакты MIDI-производства. Ответ требовался в формате Score: <1-10> и короткого объяснения. Если численная оценка не была извлечена из ответа модели, значение считалось равным 0.

Таблица 23 – Пилотная внешняя оценка DeepSeek v3.2

Модель	Средняя оценка	Медиана	Максимальная оценка
Базовая модель	1.90	2.00	6.00
Критик v1 после правил	2.80	2.00	9.00

По средней оценке дообученная модель превосходит базовую на 0.90 балла. Этот результат следует рассматривать как предварительный: объём внешней оценки мал, а отдельные ответы языковой модели могут быть нестабильны. Тем не менее он согласуется с основной гипотезой эксперимента: предварительное обучение на простых правилах может подготовить модель к последующему использованию более сложного сигнала критика.

9 Анализ ошибок и ограничений

Несмотря на положительные диагностические результаты, эксперименты выявили ряд ограничений, которые мешают получить устойчивую и крупную победу над базовой моделью. Эти ограничения важны, потому что они показывают не только слабые места конкретных запусков, но и требования к дальнейшему развитию метода.

Первая проблема связана с плотностью нот. Простое поощрение заполненности может быстро привести к переоптимизации: модель начинает генерировать слишком быстрые и перегруженные фрагменты. Формально такие примеры получают высокую оценку по компоненте плотности, но при прослушивании звучат менее естественно. Это показывает, что музыкальная “насыщенность” не сводится к числу нот. Поэтому в поздних конфигурациях максимизация плотности была заменена более осторожной наградой за умеренный диапазон значений.

Вторая проблема связана с ударными и пустыми дорожками. Базовая модель часто создаёт дополнительные дорожки, которые либо почти не содержат нот, либо состоят преимущественно из ударных событий. Это искажает как награду, так и восприятие результата: MIDI может выглядеть насыщенным, хотя содержательная мелодико-гармоническая часть остаётся слабой. Режимы с отбором информативных неударных дорожек частично решают задачу, но одновременно вносят ещё один эвристический слой в обработку результата.

Наиболее существенное ограничение связано с критиком. Он предполагает, что MIDI можно представить как две музыкальные роли: ведущую мелодию и гармоническое сопровождение. Перед оценкой выполнялась проекция исходного MIDI в формат melody/chords: сначала выбиралась наиболее мелодическая неударная дорожка по эвристикам высоты, полифонии и поступенного движения, а остальные неударные дорожки объединялись в сопровождение. Если такой вариант давал слишком мало материала, использовалась резервная проекция по событиям, где верхний голос считался мелодией, а остальные ноты — сопровождением.

Такая проекция делает вход критика совместимым с его обучающим форматом, но добавляет шум. Ошибка выбора мелодической дорожки, потеря многоголосной фактуры или неудачное разделение верхнего голоса и аккордовых нот могут изменить оценку независимо от реального качества MIDI. Поэтому запуски только с критиком нельзя трактовать как чистое измерение музыкального качества: они оценивают связку из модели, процедуры проекции и самого критика.

Отдельным недостатком оказалась чувствительность к пространству текстовых запросов. При слишком широком наборе описаний на ранних этапах обучение становится плохо интерпретируемым: модель одновременно получает сигналы по разным тональностям, размерам, стилям и фактурам. В таких условиях суммарная награда не демонстрирует устойчивого роста, а отдельные удачные пики могут быть следствием случайной генерации. Именно поэтому диагностические запуски с одним фиксированным запросом и учебная последовательность оказались более информативными.

Наконец, сохраняется высокая вычислительная стоимость экспериментов. Увеличение максимальной длины генерации даёт потенциально более полный музыкальный фрагмент, но существенно замедляет шаг обучения и может усиливать проблемы с плотностью нот. Длина 800 токенов использовалась как практический компромисс между скоростью диагностики и содержательностью результата, однако для финальной оценки музыкальности требуется проверка и на более длинных фрагментах.

10 Заключение

В работе была исследована схема последующего дообучения text-to-MIDI модели по внешним критериям качества. В отличие от обычного контролируемого дообучения, где модель оптимизирует предсказание следующего токена, использованный подход оценивает уже завершённые MIDI-фрагменты. Это позволило работать с теми свойствами, которые проявляются на уровне целой музыкальной последовательности: тональной согласованностью, ритмической устойчивостью, плотностью фактуры, долей ударных событий и общей структурной связностью.

В ходе работы был реализован полный цикл GRPO-дообучения: генерация группы кандидатов для текстового запроса, декодирование MIDI, вычисление награды, обновление LoRA-параметров и последующая диагностика качества. Были исследованы два типа обучающего сигнала. Первый основан на интерпретируемых правилах и позволяет контролировать отдельные свойства MIDI. Второй использует графовый критик, оценивающий музыкально-теоретическую согласованность фрагмента через отношения между мелодией, гармонией и метрической структурой.

Положительным результатом стало то, что в диагностических режимах модель реагировала на выбранный источник награды: rule-based конфигурации улучшали часть формальных музыкальных признаков, а использование более сильного критика после предварительного обучения на правилах дало более благоприятный результат в сравнении с базовой моделью. Это подтверждает основную гипотезу работы: последующее дообучение может смещать распределение генераций в сторону кандидатов, предпочтительных по заданному музыкальному критерию.

В то же время эксперименты показали ограничения подхода. Простые правила могут конфликтовать между собой: например, поощрение плотности нот без дополнительных ограничений приводит к чрезмерно насыщенной фактуре. Качество критика также существенно влияет на результат: слабый или смещённый критик может поощрять формальные признаки, не совпадающие с воспринимаемой музыкальностью. Кроме того, проекция произвольного MIDI в представление мелодии и аккомпанемента добавляет шум, поэтому итоговая оценка зависит не только от сгенерированного фрагмента, но и от процедуры его структурного разбора.

Дальнейшее развитие работы прежде всего требует более надёжной внешней проверки музыкального качества. Помимо автоматических оценок, нужен зрительский тест: слушателям можно предъявлять пары MIDI или аудиорендеров, полученных базовой и дообученной моделью, и просить выбрать более музыкальный, согласованный с текстовым запросом и целостный вариант. Такой тест покажет, насколько выводы по награде, A/B-сравнению и оценке языковой модели совпадают с человеческим восприятием.

Второе направление связано с улучшением критика. Перспективным является его обучение на более разнообразных парах MIDI, включение отрицательных примеров с типичными ошибками генерации и отдельная проверка согласованности оценок критика с человеческими предпочтениями. Это особенно важно, поскольку именно критик задаёт направление последующего обучения с подкреплением.

Третье направление — построение более полного пространства текстовых запросов. В диагностических экспериментах использовались узкие и контролируемые описания, что удобно для анализа обучаемости, но недостаточно для проверки обобщения. В дальнейшем следует сформировать набор запросов, покрывающий тональности, размеры, темпы, стили, инструментальные составы и типы фактуры. Такой набор позволит оценивать модель не только в узком диагностическом режиме, но и на более широком распределении задач.

Требуется также расширить набор структурных метрик для MIDI. Помимо валидности, плотности нот и доли ударных, полезно анализировать устойчивость мелодической линии, роль аккомпанемента, повторяемость мотивов, распределение регистров, согласованность голосов и соответствие инструментов текстовому описанию. Эти метрики не должны заменять прослушивание, но могут дать более подробную картину того, какие свойства модели действительно улучшаются после GRPO-дообучения.

Список использованных источников

- 1 Ji S., Yang X., Luo J. *A Survey on Deep Learning for Symbolic Music Generation: Representations, Algorithms, Evaluations, and Challenges*. *ACM Computing Surveys*. 2023. Vol. 56, no. 1. P. 1–39. DOI: <https://doi.org/10.1145/3597493>. Дата обращения: 13.05.2026.
- 2 Duan W., Yu Y., Briot J.-P. *Recent Advances in Music Generation: Methods, Evaluation, and Challenges*. *HAL preprint*. 2025. URL: <https://hal.science/hal-05452282>. Дата обращения: 13.05.2026.
- 3 Bhandari K., Roy A., Wang K., Puri G., Colton S., Herremans D. *Text2midi: Generating Symbolic Music from Captions*. *Proceedings of the AAAI Conference on Artificial Intelligence*. 2025. Vol. 39, no. 22. P. 23478–23486. DOI: <https://doi.org/10.1609/aaai.v39i22.34516>. Дата обращения: 13.05.2026.
- 4 Yuan S. et al. *ChatMusician: Understanding and Generating Music Intrinsically with Large Language Models*. *arXiv preprint*. 2024. URL: <https://arxiv.org/abs/2402.16153>. Дата обращения: 13.05.2026.
- 5 Qu X. et al. *MuPT: A Generative Symbolic Music Pretrained Transformer*. *International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=iAK9oHp4Zz>. Дата обращения: 13.05.2026.
- 6 Kumar D., Karystinaios E., Widmer G., Schedl M. *How Far Can Pretrained LLMs Go in Symbolic Music? Controlled Comparisons of Supervised and Preference-based Adaptation*. *Proceedings of the 4th Workshop on NLP for Music and Audio*. 2026. P. 27–32. DOI: <https://doi.org/10.18653/v1/2026.nlp4musa-1.5>. Дата обращения: 13.05.2026.
- 7 Briot J.-P., Hadjeres G., Pachet F. *Deep Learning Techniques for Music Generation: A Survey*. *arXiv preprint*. 2017. URL: <https://arxiv.org/abs/1709.01620>. Дата обращения: 13.05.2026.
- 8 Ji S., Luo J., Yang X. *A Comprehensive Survey on Deep Music Generation: Multi-level Representations, Algorithms, Evaluations, and Future Directions*. *arXiv preprint*. 2020. URL: <https://arxiv.org/abs/2011.06801>. Дата обращения: 13.05.2026.
- 9 Lerch A. et al. *Evaluating Generative Music Systems: A Survey*. 2025. *Survey manuscript*.
- 10 Wu S., Sun M. *Exploring the Efficacy of Pre-trained Checkpoints in Text-to-Music Generation Task*. *arXiv preprint*. 2022. URL: <https://arxiv.org/abs/2211.11216>. Дата обращения: 13.05.2026.

- 11 Huang C.-Z. A. et al. *Music Transformer: Generating Music with Long-Term Structure*. *International Conference on Learning Representations*. 2019. URL: <https://openreview.net/forum?id=rJe4ShAcF7>. Дата обращения: 13.05.2026.
- 12 Huang Y.-S., Yang Y.-H. *Pop Music Transformer: Beat-Based Modeling and Generation of Expressive Pop Piano Compositions*. *Proceedings of the ACM Multimedia Workshop on AI and Music Creativity*. 2020. URL: <https://doi.org/10.1145/3394171.3413671>. Дата обращения: 13.05.2026.
- 13 Hsiao W.-Y. et al. *Compound Word Transformer for Symbolic Music Generation*. *Proceedings of the AAAI Conference on Artificial Intelligence*. 2021. URL: <https://doi.org/10.1609/aaai.v35i1.16091>. Дата обращения: 13.05.2026.
- 14 Roberts A., Engel J., Raffel C., Hawthorne C., Eck D. *Hierarchical Latent Vector Models for Music*. *Proceedings of the International Conference on Machine Learning*. 2018. URL: <https://proceedings.mlr.press/v80/roberts18a.html>. Дата обращения: 13.05.2026.
- 15 Dong H.-W. et al. *MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation*. *Proceedings of the AAAI Conference on Artificial Intelligence*. 2018. URL: <https://doi.org/10.1609/aaai.v32i1.11312>. Дата обращения: 13.05.2026.
- 16 Mittal G. et al. *Symbolic Music Generation with Diffusion Models*. *Proceedings of the International Society for Music Information Retrieval Conference*. 2021. URL: <https://archives.ismir.net/ismir2021/paper/000020.pdf>. Дата обращения: 13.05.2026.
- 17 Plasser F. et al. *SCHmUBERT: Differentiable Symbolic Music Generation via Continuous Surrogates*. *Proceedings of the International Joint Conference on Artificial Intelligence*. 2023. URL: <https://doi.org/10.24963/ijcai.2023/674>. Дата обращения: 13.05.2026.
- 18 Huang Y. et al. *Symbolic Music Generation with Non-Differentiable Rule Guided Diffusion*. *Proceedings of Machine Learning Research*. 2024. URL: <https://proceedings.mlr.press/v235/huang24y.html>. Дата обращения: 13.05.2026.
- 19 Zeng M. et al. *MusicBERT: Symbolic Music Understanding with Large-Scale Pre-Training*. *Findings of the Association for Computational Linguistics*. 2021. URL: <https://doi.org/10.18653/v1/2021.findings-acl.70>. Дата обращения: 13.05.2026.

- 20 Yang L.-C., Lerch A. *On the Evaluation of Generative Models in Music*. *Neural Computing and Applications*. 2019. URL: <https://doi.org/10.1007/s00521-018-3849-7>. Дата обращения: 13.05.2026.
- 21 Retkowski J. et al. *Fréchet Music Distance: A Metric for Symbolic Music Generation*. 2024. *Preprint*.
- 22 Wu Y. et al. *CLAMP2: Contrastive Language-Audio Pretraining for Music and Audio*. 2025. *Preprint*.
- 23 Zhang Y., Wang Z., Wang D., Xia G. *BUTTER: A Representation Learning Framework for Bi-directional Music-Sentence Retrieval and Generation*. *Proceedings of the 1st Workshop on NLP for Music and Audio*. 2020. P. 54–58. URL: <https://aclanthology.org/2020.nlp4musa-1.8/>. Дата обращения: 13.05.2026.
- 24 Lu P. et al. *MuseCoco: Generating Symbolic Music from Text*. *arXiv preprint*. 2023. URL: <https://arxiv.org/abs/2306.00110>. Дата обращения: 13.05.2026.
- 25 Shao Z. et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. *arXiv preprint*. 2024. URL: <https://arxiv.org/abs/2402.03300>. Дата обращения: 13.05.2026.
- 26 Roy A., Puri G., Herremans D. *Text2midi-InferAlign: Improving Symbolic Music Generation with Inference-Time Alignment*. *arXiv preprint*. 2025. URL: <https://arxiv.org/abs/2505.12669>. Дата обращения: 13.05.2026.
- 27 Karystinaios E., Widmer G. *ChordGNN: Graph Neural Networks for Roman Numeral Analysis of Symbolic Music*. *Proceedings of the International Society for Music Information Retrieval Conference*. 2023. URL: <https://archives.ismir.net/ismir2023/paper/000030.pdf>. Дата обращения: 13.05.2026.
- 28 Karystinaios E., Widmer G. *Cadence Detection in Symbolic Music with Graph Neural Networks*. *Proceedings of the International Society for Music Information Retrieval Conference*. 2022. URL: <https://archives.ismir.net/ismir2022/paper/000019.pdf>. Дата обращения: 13.05.2026.
- 29 Chen T., Su L. *Functional Harmony Recognition with Multi-Task Recurrent Neural Networks*. *Proceedings of the International Society for Music Information Retrieval Conference*. 2018.

- URL: <https://archives.ismir.net/ismir2018/paper/000055.pdf>. Дата обращения: 13.05.2026.
- 30 m-a-p. *HookTheory Dataset*. *Hugging Face dataset*. URL: <https://huggingface.co/datasets/m-a-p/HookTheory>. Дата обращения: 13.05.2026.
- 31 Raffel C. *The Lakh MIDI Dataset v0.1*. URL: <https://colinraffel.com/projects/lmd/>. Дата обращения: 13.05.2026.
- 32 Kong Q., Li B., Chen J., Wang Y. *GiantMIDI-Piano: A Large-Scale MIDI Dataset for Classical Piano Music*. *Transactions of the International Society for Music Information Retrieval*. 2022. URL: <https://transactions.ismir.net/articles/10.5334/tismir.80>. Дата обращения: 13.05.2026.
- 33 Micchi G., Gotham M., Giraud M. *When in Rome: A Meta-corpus of Functional Harmony*. *Transactions of the International Society for Music Information Retrieval*. 2023. URL: <https://doi.org/10.5334/tismir.165>. Дата обращения: 13.05.2026.
- 34 Nápoles López N. *AugmentedNet: A Roman Numeral Analysis Network with Synthetic Training Examples and Additional Tonal Tasks*. 2021. URL: <https://zenodo.org/records/5624533>. Дата обращения: 13.05.2026.
- 35 DCMLab. *Distant Listening Corpus*. URL: https://github.com/DCMLab/distant_listening_corpus. Дата обращения: 13.05.2026.
- 36 De Berardinis J., Meroño-Peñuela A., Poltronieri A., Presutti V. *ChoCo: A Chord Corpus and a Data Transformation Workflow for Musical Harmony Knowledge Graphs*. *Scientific Data*. 2023. URL: <https://www.nature.com/articles/s41597-023-02410-w>. Дата обращения: 13.05.2026.
- 37 Kantarelis S., Thomas K., Lyberatos V., Dervakos E., Stamou G. *CHORDONOMICON: A Dataset of 666,000 Songs and their Chord Progressions*. *arXiv preprint*. 2024. URL: <https://arxiv.org/abs/2410.22046>. Дата обращения: 13.05.2026.
- 38 Melechovsky J., Roy A., Herremans D. *MidiCaps: A Large-Scale MIDI Dataset with Text Captions*. *arXiv preprint*. 2024. URL: <https://arxiv.org/abs/2406.02255>. Дата обращения: 13.05.2026.
- 39 Ju Z. et al. *TeleMelody: Lyric-to-Melody Generation with a Template-Based Two-Stage Method*. *arXiv preprint*. 2021. URL: <https://arxiv.org/abs/2109.09617>. Дата обращения: 13.05.2026.

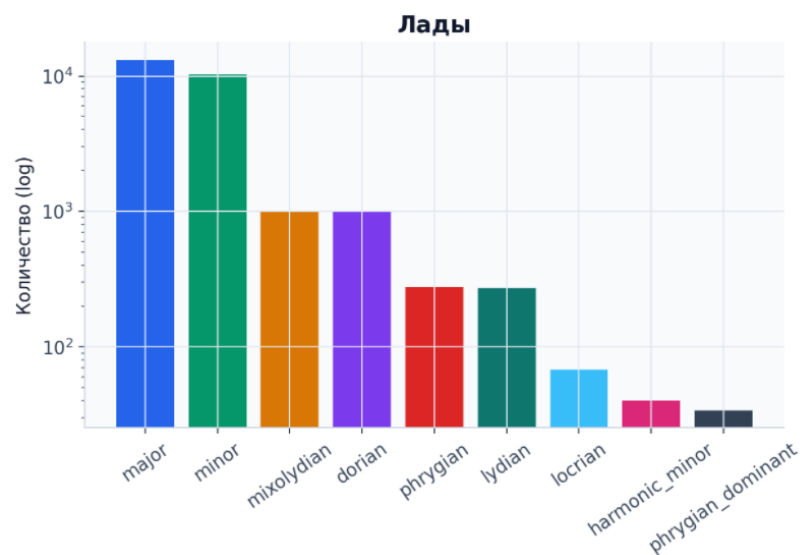
- 40 Sulun S., Davies M. E. P., Viana P. *Symbolic Music Generation Conditioned on Continuous-Valued Emotions*. *IEEE Access*. 2022. Vol. 10. P. 44617–44626. DOI: <https://doi.org/10.1109/ACCESS.2022.3169624>. Дата обращения: 13.05.2026.
- 41 abcnotation.com. *ABC Music Notation*. URL: <https://abcnotation.com/>. Дата обращения: 13.05.2026.
- 42 Wang Y. et al. *MeloTrans: A Text to Symbolic Music Generation Model Following Human Composition Habit*. *arXiv preprint*. 2024. URL: <https://arxiv.org/abs/2410.13419>. Дата обращения: 13.05.2026.
- 43 Jiang N., Jin S., Duan Z., Zhang C. *RL-Duet: Online Music Accompaniment Generation Using Deep Reinforcement Learning*. *Proceedings of the AAAI Conference on Artificial Intelligence*. 2020. URL: <https://arxiv.org/abs/2002.03082>. Дата обращения: 13.05.2026.
- 44 Cideron G. et al. *MusicRL: Aligning Music Generation to Human Preferences*. *arXiv preprint*. 2024. URL: <https://arxiv.org/abs/2402.04229>. Дата обращения: 13.05.2026.
- 45 Wang Y. et al. *NotaGen: Advancing Musicality in Symbolic Music Generation with Large Language Model Training Paradigms*. *arXiv preprint*. 2025. URL: <https://arxiv.org/abs/2502.18008>. Дата обращения: 13.05.2026.
- 46 Jonason N., Casini L., Sturm B. L. T. *SMART: Tuning a Symbolic Music Generation System with an Audio Domain Aesthetic Reward*. *arXiv preprint*. 2025. URL: <https://arxiv.org/abs/2504.16839>. Дата обращения: 13.05.2026.
- 47 Hu E. J. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. *arXiv preprint*. 2021. URL: <https://arxiv.org/abs/2106.09685>. Дата обращения: 13.05.2026.
- 48 Python Software Foundation. *Python Language Reference*. URL: <https://docs.python.org/3/reference/>. Дата обращения: 13.05.2026.
- 49 Paszke A. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. *Advances in Neural Information Processing Systems*. 2019. Vol. 32. URL: <https://papers.neurips.cc/paper/9015-pytorch>. Дата обращения: 13.05.2026.
- 50 Weights & Biases. *Weights & Biases*. URL: <https://wandb.ai/site>. Дата обращения: 13.05.2026.

51 DeepSeek-AI. *DeepSeek-V3 Technical Report*. *arXiv preprint*. 2024. URL: <https://arxiv.org/abs/2412.19437>. Дата обращения: 13.05.2026.

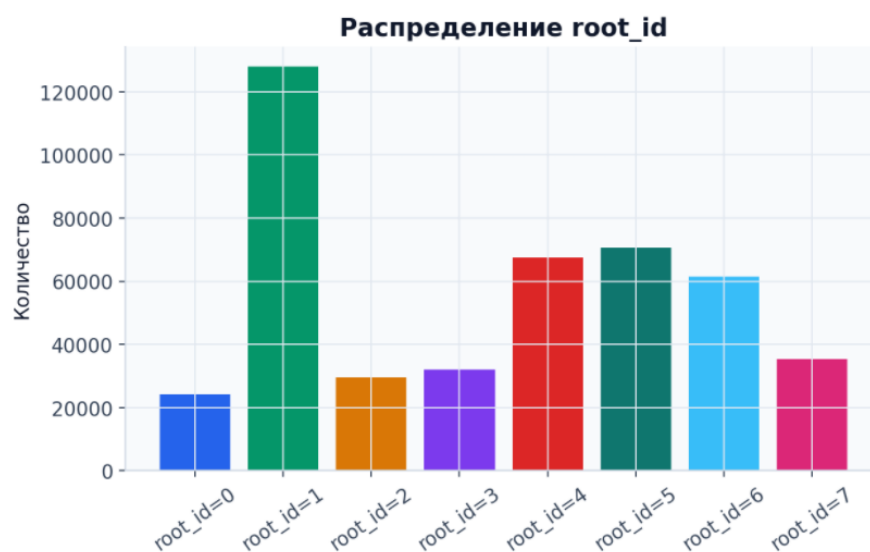
Приложение А



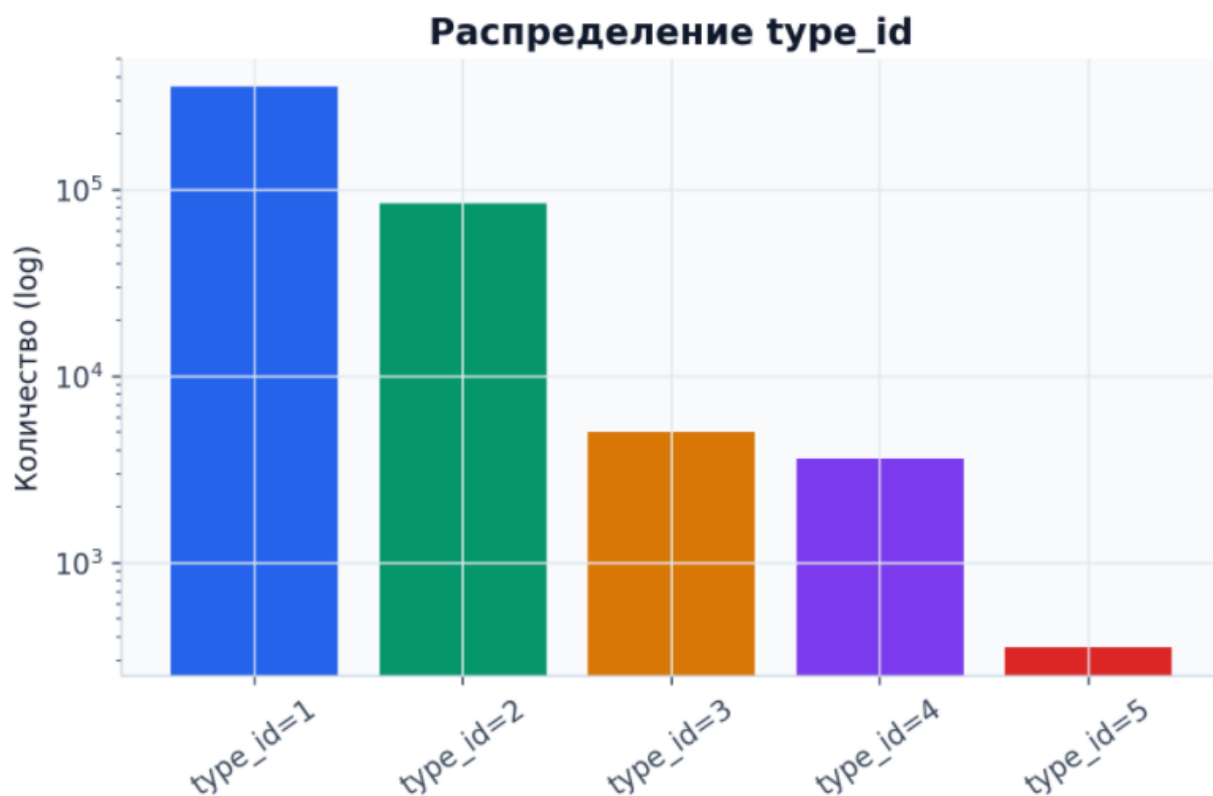
Приложение А.1 - Распределение тональностей внутри набора данных *HookTheory Dataset*



Приложение А.2 - Распределение ладов внутри набора данных *HookTheory Dataset*



Приложение А.3 - Распределение ступеней аккордов внутри набора данных *HookTheory Dataset*



Приложение А.4 - Распределение типов аккордов внутри набора данных *HookTheory Dataset*



Приложение А.5 - Распределение мелодических ступеней в наборе данных *HookTheory Dataset*